

SIMATS ENGINEERING

CO3-ASSESSMENT TOOL-1:ANALYTICAL PROBLEM SOLVING

COURSE CODE:CSA0201

COURSE NAME:C PROGRAMMING

STUDENT NAME: Shaik Mubarak

Reg No:192425194

Question 1: Student Performance Analyzer

Aim:

To develop a C program to analyze student performance using a two-dimensional array.

Algorithm

1. Start.
2. Read the number of students and subjects.
3. Input the marks of each student.
4. Calculate the total and average marks of every student.
5. Find the student with the highest average.
6. Find the highest and lowest marks in each subject.
7. Display students who failed (marks < 35 in any subject).
8. Generate the rank list based on average marks.
9. Stop.

Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n,m,i,j;
```

```
    int marks[20][20];
```

```
    int total[20]={0};
```

```
    float avg[20];
```

```
    int rank[20];
```

```
    int top=0,max,min,fail;
```

```
    printf("Enter number of students: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter number of subjects: ");
```

```

scanf("%d",&m);
printf("Enter marks:\n");
for(i=0;i<n;i++)
{
    printf("Student %d:\n",i+1);
    for(j=0;j<m;j++)
    {
        scanf("%d",&marks[i][j]);
        total[i]+=marks[i][j];
    }
    avg[i]=(float)total[i]/m;
}
// (a) Total and Average
printf("\nTotal and Average:\n");
for(i=0;i<n;i++)
{
    printf("Student %d -> Total=%d Average=%.2f\n",i+1,total[i],avg[i]);
}
// (b) Highest Average
for(i=1;i<n;i++)
{
    if(avg[i]>avg[top])
        top=i;
}
printf("\nHighest Average Student : %d\n",top+1);
// (c) Highest and Lowest in each Subject
printf("\nSubject Wise Highest and Lowest:\n");
for(j=0;j<m;j++)
{
    max=marks[0][j];
    min=marks[0][j];
    for(i=1;i<n;i++)

```

```

    {
        if(marks[i][j]>max)
            max=marks[i][j];
        if(marks[i][j]<min)
            min=marks[i][j];
    }
    printf("Subject %d -> Highest=%d Lowest=%d\n",j+1,max,min);
}

// (d) Failed Students
printf("\nFailed Students:\n");
fail=0;
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        if(marks[i][j]<35)
        {
            printf("Student %d\n",i+1);
            fail=1;
            break;
        }
    }
}

if(fail==0)
    printf("No Failed Students\n");

// (e) Rank List
for(i=0;i<n;i++)
    rank[i]=1;
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {

```

```

        if(avg[j]>avg[i])
            rank[i]++;
    }
}
printf("\nRank List:\n");
for(i=0;i<n;i++)
{
    printf("Student %d -> Rank %d\n",i+1,rank[i]);
}
return 0;
}

```

Output

```

Output
Enter number of students: 3
Enter number of subjects: 3
Enter marks:
Student 1:
85 95 90
Student 2:
96 56 25
Student 3:
75 85 95

Total and Average:
Student 1 -> Total=270 Average=90.00
Student 2 -> Total=177 Average=59.00
Student 3 -> Total=255 Average=85.00

Highest Average Student : 1

Subject Wise Highest and Lowest:
Subject 1 -> Highest=96 Lowest=75
Subject 2 -> Highest=95 Lowest=56
Subject 3 -> Highest=95 Lowest=25

Failed Students:
Student 2

Rank List:
Student 1 -> Rank 1
Student 2 -> Rank 3
Student 3 -> Rank 2

=== Code Execution Successful ===

```

Question 2: Hospital Patient Queue Analyzer

Aim

To develop a C program to manage hospital patient records using arrays and perform queue analysis based on age and priority.

Program

```
#include <stdio.h>

int main()
{
    int n, i, id[20], age[20], priority[20];
    char name[20][30];
    int search, maxAge, minAge, oldest, youngest;
    int sum = 0;
    float avg;
    printf("Enter number of patients: ");
    scanf("%d", &n);
    printf("\nEnter Patient Details:\n");
    for(i = 0; i < n; i++)
    {
        printf("\nPatient %d\n", i + 1);
        printf("ID: ");
        scanf("%d", &id[i]);
        printf("Name: ");
        scanf("%s", name[i]);
        printf("Age: ");
        scanf("%d", &age[i]);
        printf("Priority (1-High, 2-Medium, 3-Low): ");
        scanf("%d", &priority[i]);
        sum += age[i];
    }
    // (a) Search patient by ID
    printf("\nEnter Patient ID to Search: ");
    scanf("%d", &search);
```

```

for(i = 0; i < n; i++)
{
    if(id[i] == search)
    {
        printf("\nPatient Found\n");
        printf("ID: %d\n", id[i]);
        printf("Name: %s\n", name[i]);
        printf("Age: %d\n", age[i]);
        printf("Priority: %d\n", priority[i]);
        break;
    }
}

if(i == n)
    printf("Patient Not Found\n");

// (b) Sort by Priority
printf("\nPatients Sorted by Priority:\n");
for(i = 1; i <= 3; i++)
{
    int j;
    for(j = 0; j < n; j++)
    {
        if(priority[j] == i)
        {
            printf("%d %s Age:%d Priority:%d\n",
                id[j], name[j], age[j], priority[j]);
        }
    }
}

// (c) Oldest and Youngest
oldest = youngest = 0;
for(i = 1; i < n; i++)
{

```

```

        if(age[i] > age[oldest])
            oldest = i;
        if(age[i] < age[youngest])
            youngest = i;
    }
    printf("\nOldest Patient: %s (%d years)\n",
        name[oldest], age[oldest]);
    printf("Youngest Patient: %s (%d years)\n",
        name[youngest], age[youngest]);
    // (d) Immediate Attention
    printf("\nPatients Requiring Immediate Attention:\n");
    for(i = 0; i < n; i++)
    {
        if(priority[i] == 1)
            printf("%d %s\n", id[i], name[i]);
    }
    // (e) Average Age
    avg = (float)sum / n;
    printf("\nAverage Age = %.2f\n", avg);
    return 0;
}

```

Output

```
main.c
1 #include <stdio.h>
2 int main()
3 {
4     int n, i, id[20], age[20], priority[20];
5     char name[20][30];
6     int search, minAge, maxAge, oldest, youngest;
7     int sum = 0;
8     float avg;
9     printf("Enter number of patients: ");
10    scanf("%d", &n);
11    printf("\nEnter Patient Details:\n");
12    for(k = 0; k < n; k++)
13    {
14        printf("Enter Patient %d\n", k + 1);
15        printf("ID: ");
16        scanf("%d", &id[k]);
17        printf("Name: ");
18        scanf("%s", name[k]);
19        printf("Age: ");
20        scanf("%d", &age[k]);
21        printf("Priority (1-High, 2-Medium, 3-Low): ");
22        scanf("%d", &priority[k]);
23        sum += age[k];
24    }
25    // (a) Search patient by ID
26    printf("\nEnter Patient ID to Search: ");
27    scanf("%d", &search);
28    for(i = 0; i < n; i++)
29    {
30        if(id[i] == search)
31        {
32            printf("Patient Found:\n");
33            printf("ID: %d\n", id[i]);
34            printf("Name: %s\n", name[i]);
35            printf("Age: %d\n", age[i]);
36            printf("Priority: %d\n", priority[i]);
37            break;
38        }
39    }
40    if(i == n)
41        printf("Patient Not Found\n");
42    // (b) Sort by Priority
```

```
Output
Enter number of patients: 2
Enter Patient Details:
Patient 1
ID: 102425
Name: ashe
Age: 18
Priority (1-High, 2-Medium, 3-Low): 2
Patient 2
ID: 102426
Name: Shaik
Age: 20
Priority (1-High, 2-Medium, 3-Low): 1
Enter Patient ID to Search: 102426
Patient Found
ID: 102426
Name: Shaik
Age: 20
Priority: 1
Patients Sorted by Priority:
102426 Shaik Age:20 Priority:1
102425 ashe Age:18 Priority:2
Oldest Patient: Shaik (20 years)
Youngest Patient: ashe (18 years)
Patients Requiring Immediate Attention:
102426 Shaik
Average Age = 19.00
--- Code Execution Successful ---
```

Question 3: Sales Performance Tracker

Aim

To develop a C program to analyze monthly sales data of products using a two-dimensional array.

Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j;
```

```
    float sales[20][12];
```

```
    float total[20] = {0}, avg[20];
```

```
    int best = 0, month;
```

```
    float max, grandTotal = 0;
```

```
    printf("Enter number of products: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter monthly sales for each product:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```



```

printf("\nProduct %d\n", i + 1);
for(j = 0; j < 12; j++)
{
    printf("Month %d: ", j + 1);
    scanf("%f", &sales[i][j]);
    total[i] += sales[i][j];
}
avg[i] = total[i] / 12;
grandTotal += total[i];
}
// (a) Annual sales
printf("\nAnnual Sales of Products:\n");
for(i = 0; i < n; i++)
    printf("Product %d = %.2f\n", i + 1, total[i]);
// (b) Best selling product
for(i = 1; i < n; i++)
{
    if(total[i] > total[best])
        best = i;
}
printf("\nBest Selling Product: Product %d\n", best + 1);
// (c) Month with highest sales
max = sales[0][0];
month = 1;
for(j = 0; j < 12; j++)
{
    float sum = 0;
    for(i = 0; i < n; i++)
        sum += sales[i][j];
    if(j == 0 || sum > max)
    {
        max = sum;

```

```

        month = j + 1;
    }
}
printf("\nMonth with Highest Sales: Month %d\n", month);
// (d) Products with continuously decreasing sales
printf("\nProducts with Continuously Decreasing Sales:\n");
int found = 0;
for(i = 0; i < n; i++)
{
    int flag = 1;
    for(j = 0; j < 11; j++)
    {
        if(sales[i][j] < sales[i][j + 1])
        {
            flag = 0;
            break;
        }
    }
    if(flag)
    {
        printf("Product %d\n", i + 1);
        found = 1;
    }
}
if(found == 0)
    printf("None\n");
// (e) Products above average annual sales
float overallAvg = grandTotal / n;
printf("\nProducts Above Overall Average Sales:\n");
for(i = 0; i < n; i++)
{
    if(total[i] > overallAvg)

```

```

        printf("Product %d\n", i + 1);
    }
    return 0;
}

```

Output

The screenshot shows a C program being executed in an IDE. The code on the left calculates monthly and annual sales for two products, identifies the best-selling product, and finds the month with the highest sales. The output on the right shows the user input and the program's results.

```

main.c
1 #include <stdio.h>
2 int main()
3 {
4     int n, i, j;
5     float sales[20][12];
6     float total[20] = {0}, avg[20];
7     int best = 0, month;
8     float max, grandTotal = 0;
9     printf("Enter number of products: ");
10    scanf("%d", &n);
11    printf("Enter monthly sales for each product:\n");
12    for(i = 0; i < n; i++)
13    {
14        printf("\nProduct %d\n", i + 1);
15        for(j = 0; j < 12; j++)
16        {
17            printf("Month %d: ", j + 1);
18            scanf("%d", &sales[i][j]);
19            total[i] += sales[i][j];
20        }
21        avg[i] = total[i] / 12;
22        grandTotal += total[i];
23    }
24    // (a) Annual sales
25    printf("\nAnnual Sales of Products:\n");
26    for(i = 0; i < n; i++)
27        printf("Product %d = %.2f\n", i + 1, total[i]);
28    // (b) Best selling product
29    for(i = 1; i < n; i++)
30    {
31        if(total[i] > total[best])
32            best = i;
33    }
34    printf("\nBest Selling Product: Product %d\n", best + 1);
35
36    month = 1;
37    for(j = 0; j < 12; j++)
38    {
39        float sum = 0;
40        for(i = 0; i < n; i++)
41            sum += sales[i][j];
42        if(j == 0 || sum > max)
43        {
44            max = sum;
45            month = j + 1;
46        }
47    }
48    printf("\nMonth with Highest Sales: Month %d\n", month);
49    // (d) Products with continuously decreasing sales
50    printf("\nProducts with Continuously Decreasing Sales:\n");
51    int found = 0;
52    for(i = 0; i < n; i++)
53    {
54

```

```

Output
Enter number of products: 2
Enter monthly sales for each product:

Product 1
Month 1: 200
Month 2: 56
Month 3: 35
Month 4: 26
Month 5: 12
Month 6: 45
Month 7: 64
Month 8: 55
Month 9: 2
Month 10: 3
Month 11: 1
Month 12: 3

Product 2
Month 1: 25
Month 2: 56
Month 3: 41
Month 4: 10
Month 5: 24
Month 6: 58
Month 7: 65
Month 8: 48
Month 9: 7
Month 10: 15
Month 11: 98
Month 12: 46

Annual Sales of Products:
Product 1 = 502.00
Product 2 = 493.00

Best Selling Product: Product 1

Month with Highest Sales: Month 1

Products with Continuously Decreasing Sales:
None

Products Above Overall Average Sales:
Product 1

--- Code Execution Successful ---

```

Question 4: Number Classification and Analysis

Aim

To develop a C program to classify integers into positive, negative, and zero categories and perform statistical analysis on them.

Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```

int n, i;
int a[100];
int pos[100], neg[100], zero[100];
int p = 0, ne = 0, z = 0;
int sumPos = 0, sumNeg = 0;
int maxPos, minNeg;
int posFound = 0, negFound = 0;
printf("Enter number of elements: ");
scanf("%d", &n);
printf("Enter the elements:\n");
for(i = 0; i < n; i++)
{
    scanf("%d", &a[i]);
    if(a[i] > 0)
    {
        pos[p++] = a[i];
        sumPos += a[i];
        if(!posFound || a[i] > maxPos)
            maxPos = a[i];
        posFound = 1;
    }
    else if(a[i] < 0)
    {
        neg[ne++] = a[i];
        sumNeg += a[i];
        if(!negFound || a[i] < minNeg)
            minNeg = a[i];
        negFound = 1;
    }
    else
    {
        zero[z++] = a[i];
    }
}

```

```

    }
}
// (a) Separate positive, negative and zero
printf("\nPositive Numbers:\n");
for(i = 0; i < p; i++)
    printf("%d ", pos[i]);
printf("\n\nNegative Numbers:\n");
for(i = 0; i < ne; i++)
    printf("%d ", neg[i]);
printf("\n\nZero Values:\n");
for(i = 0; i < z; i++)
    printf("%d ", zero[i]);
// (b) Largest positive and smallest negative
if(posFound)
    printf("\n\nLargest Positive = %d", maxPos);
else
    printf("\n\nNo Positive Numbers");
if(negFound)
    printf("\n\nSmallest Negative = %d", minNeg);
else
    printf("\n\nNo Negative Numbers");
// (c) Sum of positive and negative numbers
printf("\n\nSum of Positive Numbers = %d", sumPos);
printf("\n\nSum of Negative Numbers = %d", sumNeg);
// (d) Category with maximum count
if(p >= ne && p >= z)
    printf("\nCategory with Maximum Elements: Positive");
else if(ne >= p && ne >= z)
    printf("\nCategory with Maximum Elements: Negative");
else
    printf("\nCategory with Maximum Elements: Zero");
// (e) Count of each category

```

```

printf("\n\nPositive Count = %d", p);
printf("\n\nNegative Count = %d", ne);
printf("\n\nZero Count = %d", z);
return 0;
}

```

Output

The screenshot shows a C program in a code editor with the following code:

```

1 #include <stdio.h>
2 int main()
3 {
4     int n, i;
5     int a[100];
6     int pos[100], neg[100], zero[100];
7     int p = 0, ne = 0, z = 0;
8     int sumPos = 0, sumNeg = 0;
9     int maxPos, minNeg;
10    int posFound = 0, negFound = 0;
11    printf("Enter number of elements: ");
12    scanf("%d", &n);
13    printf("Enter the elements:\n");
14    for(i = 0; i < n; i++)
15    {
16        scanf("%d", &a[i]);
17        if(a[i] > 0)
18        {
19            pos[p++] = a[i];
20            sumPos += a[i];
21            if(!posFound || a[i] > maxPos)
22                maxPos = a[i];
23            posFound = 1;
24        }
25        else if(a[i] < 0)
26        {
27            neg[ne++] = a[i];
28            sumNeg += a[i];
29            if(!negFound || a[i] < minNeg)
30                minNeg = a[i];
31            negFound = 1;
32        }
33        else

```

The output window shows the following results:

```

Enter number of elements: 3
Enter the elements:
15 -8 56

Positive Numbers:
15 56

Negative Numbers:
-8

Zero Values:

Largest Positive = 56
Smallest Negative = -8

Sum of Positive Numbers = 71
Sum of Negative Numbers = -8
Category with Maximum Elements: Positive

Positive Count = 2
Negative Count = 1
Zero Count = 0

--- Code Execution Successful ---

```

Question 5: Library Book Inventory System

Aim

To develop a C program to manage a library book inventory using arrays and perform search, availability, and analysis operations.

Program

```

#include <stdio.h>
#include <string.h>

int main()
{
    int n, i, searchId;
    int id[50], copies[50], issued[50];
    char title[50][50];
    printf("Enter number of books: ");

```

```

scanf("%d", &n);
printf("\nEnter Book Details:\n");
for(i = 0; i < n; i++)
{
    printf("\nBook %d\n", i + 1);
    printf("Book ID: ");
    scanf("%d", &id[i]);
    printf("Title: ");
    scanf("%s", title[i]);
    printf("Available Copies: ");
    scanf("%d", &copies[i]);
    printf("Issued Copies: ");
    scanf("%d", &issued[i]);
}
// (a) Search Book by ID
printf("\nEnter Book ID to Search: ");
scanf("%d", &searchId);
int found = 0;
for(i = 0; i < n; i++)
{
    if(id[i] == searchId)
    {
        printf("\nBook Found\n");
        printf("ID : %d\n", id[i]);
        printf("Title : %s\n", title[i]);
        printf("Available : %d\n", copies[i]);
        printf("Issued : %d\n", issued[i]);
        found = 1;
        break;
    }
}
if(found == 0)

```

```

    printf("Book Not Found\n");
// (b) Books Out of Stock
printf("\nBooks Out of Stock:\n");
found = 0;
for(i = 0; i < n; i++)
{
    if(copies[i] == 0)
    {
        printf("%s\n", title[i]);
        found = 1;
    }
}
if(found == 0)
    printf("None\n");
// (c) Most Issued Book
int max = 0;
for(i = 1; i < n; i++)
{
    if(issued[i] > issued[max])
        max = i;
}
printf("\nMost Issued Book: %s\n", title[max]);
// (d) Total Available Copies
int total = 0;
for(i = 0; i < n; i++)
    total += copies[i];
printf("\nTotal Available Copies = %d\n", total);
// (e) Books with More Than 5 Copies
printf("\nBooks Having More Than 5 Copies:\n");
found = 0;
for(i = 0; i < n; i++)
{

```



```

        if(copies[i] > 5)
        {
            printf("%s\n", title[i]);
            found = 1;
        }
    }
    if(found == 0)
        printf("None\n");

    return 0;
}

```

Output

The screenshot shows a C program running in a terminal. The program manages a book inventory with 50 slots. It prompts the user to enter the number of books (2), then for each book, it asks for ID, title, available copies, and issued copies. After input, it prompts for a search ID (1924). The program then displays the found book details, books out of stock, the most issued book, total available copies, and books with more than 5 copies.

```

main.c
1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     int n, i, searchId;
6     int id[50], copies[50], issued[50];
7     char title[50][50];
8     printf("Enter number of books: ");
9     scanf("%d", &n);
10    printf("\nEnter Book Details:\n");
11    for(i = 0; i < n; i++)
12    {
13        printf("\nBook %d\n", i + 1);
14        printf("Book ID: ");
15        scanf("%d", &id[i]);
16        printf("Title: ");
17        scanf("%s", title[i]);
18        printf("Available Copies: ");
19        scanf("%d", &copies[i]);
20        printf("Issued Copies: ");
21        scanf("%d", &issued[i]);
22    }
23    // (a) Search Book by ID
24    printf("\nEnter Book ID to Search: ");
25    scanf("%d", &searchId);
26    int found = 0;
27    for(i = 0; i < n; i++)
28    {
29        if(id[i] == searchId)
30        {
31            printf("\nBook Found\n");
32            printf("ID : %d\n", id[i]);
33            printf("Title : %s\n", title[i]);
34            printf("Available : %d\n", copies[i]);
35            printf("Issued : %d\n", issued[i]);
36            found = 1;
37            break;

```

Output

```

Enter number of books: 2

Enter Book Details:

Book 1
Book ID: 1924
Title: Stranger
Available Copies: 45
Issued Copies: 44

Book 2
Book ID: 1925
Title: Wednesday
Available Copies: 0
Issued Copies: 25

Enter Book ID to Search: 1924

Book Found
ID : 1924
Title : Stranger
Available : 45
Issued : 44

Books Out of Stock:
Wednesday

Most Issued Book: Stranger

Total Available Copies = 45

Books Having More Than 5 Copies:
Stranger

--- Code Execution Successful ---

```

Question 6: Sparse Matrix Analyzer

Aim

To develop a C program to determine whether a matrix is sparse and perform matrix analysis operations.

Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```

int r,c,i,j;
int a[10][10];
int zero=0,nonzero=0,maxRow=0,maxCount=0;
printf("Enter rows and columns: ");
scanf("%d%d",&r,&c);
printf("Enter matrix elements:\n");
for(i=0;i<r;i++)
{
    for(j=0;j<c;j++)
    {
        scanf("%d",&a[i][j]);
        if(a[i][j]==0)
            zero++;
        else
            nonzero++;
    }
}
printf("\nZero Elements = %d",zero);
printf("\nNon-Zero Elements = %d",nonzero);
printf("\nPercentage of Zero Elements = %.2f%%\n",
    (float)zero*100/(r*c));
if(zero > (r*c)/2)
{
    printf("\nMatrix is Sparse Matrix\n");
    printf("\nTriplet Representation:\n");
    printf("Row Col Value\n");
    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            if(a[i][j]!=0)
                printf("%d %d %d\n",i,j,a[i][j]);
        }
    }
}

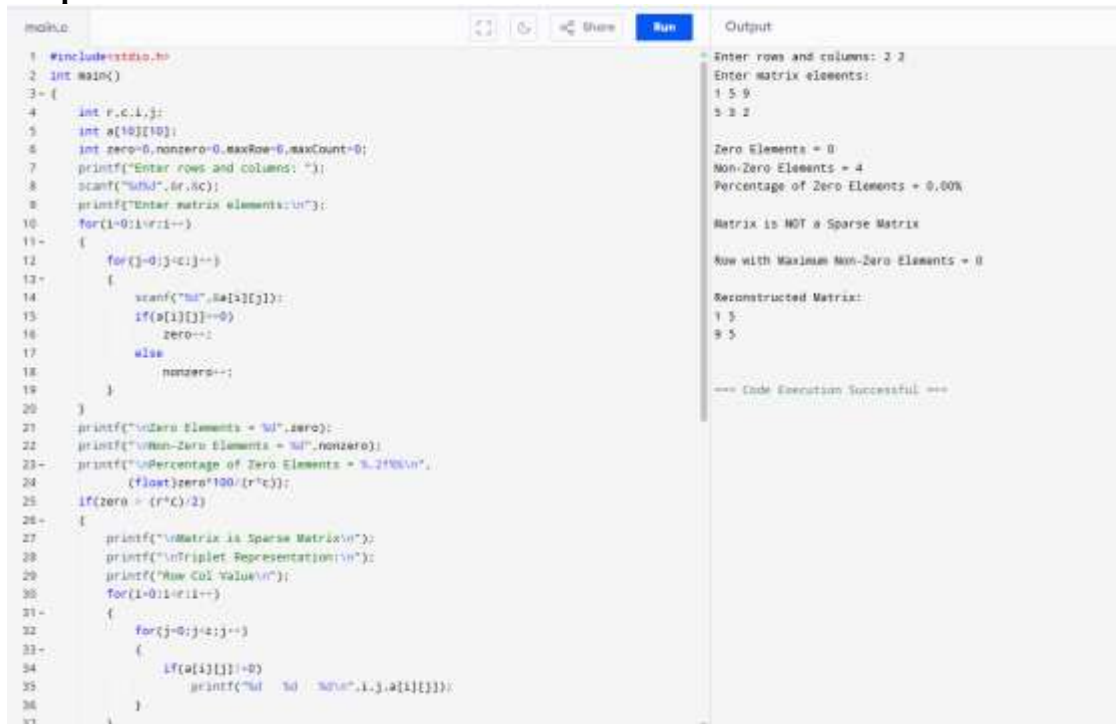
```

```

        }
    }
}
else
{
    printf("\nMatrix is NOT a Sparse Matrix\n");
}
for(i=0;i<r;i++)
{
    int count=0;
    for(j=0;j<c;j++)
    {
        if(a[i][j]!=0)
            count++;
    }
    if(count>maxCount)
    {
        maxCount=count;
        maxRow=i;
    }
}
printf("\nRow with Maximum Non-Zero Elements = %d\n",maxRow);
printf("\nReconstructed Matrix:\n");
for(i=0;i<r;i++)
{
    for(j=0;j<c;j++)
        printf("%d ",a[i][j]);
    printf("\n");
}
return 0;
}

```

Output



The screenshot shows a C program in a code editor with a 'Run' button and an 'Output' window. The code defines a 3x3 matrix and checks if it is a sparse matrix. The output window shows the following text:

```
Enter rows and columns: 3 3
Enter matrix elements:
1 5 9
5 3 2

Zero Elements = 0
Non-Zero Elements = 4
Percentage of Zero Elements = 0.00%

Matrix is NOT a Sparse Matrix
Row with Maximum Non-Zero Elements = 0

Reconstructed Matrix:
1 5
9 5

--- Code Execution Successful ---
```

Question 7: Bank Transaction Fraud Analyzer

Aim

To develop a C program to analyze bank transactions and detect withdrawals, repeated transactions, and calculate the final account balance.

Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j;
```

```
    float trans[100];
```

```
    float deposit=0,withdraw=0,balance;
```

```
    float initial,limit;
```

```
    int found=0;
```

```
    printf("Enter initial balance: ");
```

```
    scanf("%f",&initial);
```

```
    printf("Enter number of transactions: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter transactions (+Deposit, -Withdrawal):\n");
```

```

for(i=0;i<n;i++)
    scanf("%f",&trans[i]);
// (a) Total Deposits and Withdrawals
for(i=0;i<n;i++)
{
    if(trans[i]>0)
        deposit+=trans[i];
    else
        withdraw+=-trans[i];
}
printf("\nTotal Deposits = %.2f",deposit);
printf("\nTotal Withdrawals = %.2f\n",withdraw);
// (b) Largest Withdrawal
float largest=0;
for(i=0;i<n;i++)
{
    if(trans[i]<0)
    {
        if(-trans[i]>largest)
            largest=-trans[i];
    }
}
printf("\nLargest Withdrawal = %.2f\n",largest);
// (c) Consecutive Withdrawals
printf("\nEnter withdrawal limit: ");
scanf("%f",&limit);
printf("\nConsecutive Withdrawals Exceeding Limit:\n");
found=0;
for(i=0;i<n-1;i++)
{
    if(trans[i]<0 && trans[i+1]<0)
    {

```

```

        if((-trans[i])+(-trans[i+1])>limit)
        {
            printf("%.2f %.2f\n",trans[i],trans[i+1]);
            found=1;
        }
    }
}

if(found==0)
    printf("None\n");

// (d) Repeated Transactions
printf("\nRepeated Transactions:\n");
found=0;
for(i=0;i<n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(trans[i]==trans[j])
        {
            printf("%.2f\n",trans[i]);
            found=1;
            break;
        }
    }
}

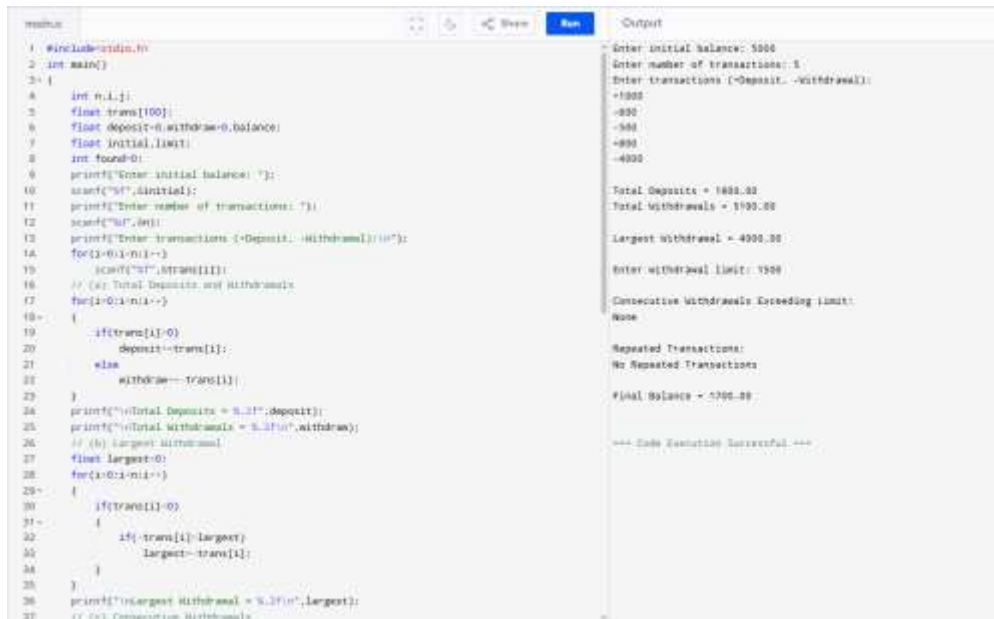
if(found==0)
    printf("No Repeated Transactions\n");

// (e) Final Balance
balance=initial;
for(i=0;i<n;i++)
    balance+=trans[i];
printf("\nFinal Balance = %.2f\n",balance);
return 0;

```

}

Output



The screenshot shows a C program for a bank transaction system. The source code on the left includes headers, defines arrays for transactions, and implements logic for deposits, withdrawals, and limit checks. The output on the right shows the program's execution with user inputs and calculated values.

```
#include<stdio.h>
int main()
{
    int n,i,j;
    float trans[100];
    float deposit=0,withdraw=0,balance;
    float initial,limit;
    int found=0;
    printf("Enter initial balance: ");
    scanf("%f",&initial);
    printf("Enter number of transactions: ");
    scanf("%d",&n);
    printf("Enter transactions (-Deposit, +Withdrawal):\n");
    for(i=0;i<n;i++)
    {
        scanf("%f",&trans[i]);
        // (x) Total Deposits and Withdrawals
        for(j=0;j<n;j++)
        {
            if(trans[j]<0)
                deposit+=trans[j];
            else
                withdraw+=trans[j];
        }
        printf("\nTotal Deposits = %.2f",deposit);
        printf("\nTotal Withdrawals = %.2f",withdraw);
        // (b) Largest Withdrawal
        float largest=0;
        for(x=0;x<n;x++)
        {
            if(trans[x]<0)
            {
                if(-trans[x]>largest)
                    largest=-trans[x];
            }
        }
        printf("\nLargest Withdrawal = %.2f",largest);
        // (c) Consecutive Withdrawals
    }
}
```

Output:

```
Enter initial balance: 5000
Enter number of transactions: 5
Enter transactions (-Deposit, +Withdrawal):
+1000
-500
-500
+900
-4000
Total Deposits = 1000.00
Total Withdrawals = 5000.00
Largest Withdrawal = 4000.00
Enter withdrawal limit: 1500
Consecutive Withdrawals Exceeding limit:
None
Repeated Transactions:
No Repeated Transactions
Final Balance = 5000.00
--- Code Execution Successful ---
```

Question 8: String Similarity and Pattern Analyzer

Aim

To develop a C program to compare two strings and analyze their similarity.

Program

```
#include<stdio.h>
#include<string.h>

int main()
{
    char s1[100], s2[100];
    int i, j;
    int f1[26]={0}, f2[26]={0};
    int common=0, prefix=0;
    float similarity;
    printf("Enter First String: ");
    scanf("%s", s1);
    printf("Enter Second String: ");
    scanf("%s", s2);
    // Frequency of characters
```

```

for(i=0;s1[i]!='\0';i++)
    if(s1[i]>='a' && s1[i]<='z')
        f1[s1[i]-'a']++;
for(i=0;s2[i]!='\0';i++)
    if(s2[i]>='a' && s2[i]<='z')
        f2[s2[i]-'a']++;
// (a) Check Anagram
int anagram=1;
for(i=0;i<26;i++)
{
    if(f1[i]!=f2[i])
    {
        anagram=0;
        break;
    }
}
if(anagram)
    printf("\nStrings are Anagrams\n");
else
    printf("\nStrings are Not Anagrams\n");
// (b) Count Common Characters
for(i=0;i<26;i++)
{
    if(f1[i]>0 && f2[i]>0)
        common++;
}
printf("\nNumber of Common Characters = %d\n", common);
// (c) Longest Common Prefix
while(s1[prefix]==s2[prefix] && s1[prefix]!='\0')
    prefix++;
printf("\nLongest Common Prefix: ");
for(i=0;i<prefix;i++)

```



```

        printf("%c",s1[i]);
    if(prefix==0)
        printf("None");
    // (d) Characters in Both Strings with Frequency
    printf("\n\nCommon Characters and Frequencies:\n");
    for(i=0;i<26;i++)
    {
        if(f1[i]>0 && f2[i]>0)
        {
            printf("%c -> %d , %d\n",i+'a',f1[i],f2[i]);
        }
    }
    // (e) Similarity Percentage
    int total = strlen(s1)+strlen(s2);
    similarity = (2.0*common*100)/total;
    printf("\nSimilarity Percentage = %.2f%%\n", similarity);
    return 0;
}

```

Output

The screenshot shows a C program being executed in an IDE. The code on the left implements an anagram checker and similarity percentage calculator. It takes two strings, 'Shaik' and 'Mubarak', as input. The output on the right shows the program's execution results, including the input strings, a confirmation that they are not anagrams, the count of common characters (2), the longest common prefix (None), the frequency of common characters (a: 1, k: 1), and the similarity percentage (33.33%).

```

main.c
1 #include<stdio.h>
2 #include<string.h>
3 int main()
4 {
5     char s1[100], s2[100];
6     int i, j;
7     int f1[26]={0}, f2[26]={0};
8     int common=0, prefix=0;
9     float similarity;
10    printf("Enter First String: ");
11    scanf("%s", s1);
12    printf("Enter Second String: ");
13    scanf("%s", s2);
14    // Frequency of characters
15    for(i=0;s1[i]!='\0';i++)
16        if(s1[i]!='a' && s1[i]!='z')
17            f1[s1[i]-'a']++;
18    for(i=0;s2[i]!='\0';i++)
19        if(s2[i]!='a' && s2[i]!='z')
20            f2[s2[i]-'a']++;
21    // (a) Check Anagram
22    int anagram=1;
23    for(i=0;i<26;i++)
24    {
25        if(f1[i]!=f2[i])
26        {
27            anagram=0;
28            break;
29        }
30    }
31    if(anagram)

```

Output

```

Enter First String: Shaik
Enter Second String: Mubarak

Strings are Not Anagrams

Number of Common Characters = 2

Longest Common Prefix: None

Common Characters and Frequencies:
a -> 1 , 2
k -> 1 , 1

Similarity Percentage = 33.33%

--- Code Execution Successful ---

```

Question 9: Dynamic Seat Allocation System

Aim

To develop a C program to manage seat allocation in a theatre using a two-dimensional array.

Program

```
#include<stdio.h>

int main()
{
    int n,m,i,j,k;
    int seat[20][20];
    int available=0, booked=0;
    int row=-1,start=-1,maxRow=0,maxAvail=0;
    printf("Enter number of rows: ");
    scanf("%d",&n);
    printf("Enter number of seats per row: ");
    scanf("%d",&m);
    printf("Enter seat arrangement (0-Available, 1-Booked):\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            scanf("%d",&seat[i][j]);
            if(seat[i][j]==0)
                available++;
            else
                booked++;
        }
    }

    // (a) Display arrangement
    printf("\nSeat Arrangement:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
```

```

        printf("%d ",seat[i][j]);
    printf("\n");
}
// (b) Find first block of K consecutive seats
printf("\nEnter K (required consecutive seats): ");
scanf("%d",&k);
for(i=0;i<n;i++)
{
    int count=0;
    for(j=0;j<m;j++)
    {
        if(seat[i][j]==0)
            count++;
        else
            count=0;
        if(count==k)
        {
            row=i;
            start=j-k+1;
            break;
        }
    }
    if(row!=-1)
        break;
}
if(row!=-1)
    printf("\nFirst Available Block: Row %d, Seat %d to %d\n",
        row+1,start+1,start+k);
else
    printf("\nNo Consecutive Seats Available\n");
// (c) Book selected seats
if(row!=-1)

```

```

{
    for(j=start;j<start+k;j++)
        seat[row][j]=1;
    printf("\nSeats Booked Successfully.\n");
}
// (d) Occupancy Percentage
float occupancy=(float)booked*100/(n*m);
printf("\nOccupancy Percentage = %.2f%%\n",occupancy);
// (e) Row with maximum available seats
for(i=0;i<n;i++)
{
    int count=0;
    for(j=0;j<m;j++)
    {
        if(seat[i][j]==0)
            count++;
    }
    if(count>maxAvail)
    {
        maxAvail=count;
        maxRow=i;
    }
}
printf("\nRow with Maximum Available Seats = %d\n",maxRow+1);
return 0;
}

```

Output

```
main.c 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 Run Output
```

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,m,i,j,k;
5     int seat[20][20];
6     int available=0, booked=0;
7     int row=1, start=1, maxRow=8, maxAvail=0;
8     printf("Enter number of rows: ");
9     scanf("%d",&n);
10    printf("Enter number of seats per row: ");
11    scanf("%d",&m);
12    printf("Enter seat arrangement (0-Available, 1-Booked):\n");
13    for(i=0;i<n;i++)
14    {
15        for(j=0;j<m;j++)
16        {
17            scanf("%d",&seat[i][j]);
18            if(seat[i][j]==0)
19                available++;
20            else
21                booked++;
22        }
23    }
24    // (a) Display arrangement
25    printf("\nSeat Arrangement:\n");
26    for(i=0;i<n;i++)
27    {
28        for(j=0;j<m;j++)
29            printf("%d ",seat[i][j]);
30        printf("\n");
31    }
32    // (b) Find first block of K consecutive seats
33    printf("\nEnter K (required consecutive seats): ");
34    scanf("%d",&k);
35    for(i=0;i<n;i++)
36    {
37        int count=0;
```

```
Enter number of rows: 3
Enter number of seats per row: 2
Enter seat arrangement (0-Available, 1-Booked):
1 0 1
1 1 1

Seat Arrangement:
1 0
1 1
1 1

Enter K (required consecutive seats): 2
No Consecutive Seats Available
Occupancy Percentage = 83.33%
Row with Maximum Available Seats = 1

--- Code Execution Successful ---
```

Question 10: Electricity Consumption Analyzer

Aim

To develop a C program to analyze the electricity consumption of households for 12 months using a two-dimensional array.

Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j;
```

```
    float units[20][12];
```

```
    float total[20]={0};
```

```
    float overall=0;
```

```
    int maxHouse=0,minHouse=0;
```

```
    int maxMonth=0;
```

```
    float monthTotal,maxTotal=0;
```

```
    printf("Enter number of households: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter electricity consumption for 12 months:\n");
```

```

for(i=0;i<n;i++)
{
    printf("\nHousehold %d:\n",i+1);
    for(j=0;j<12;j++)
    {
        scanf("%f",&units[i][j]);
        total[i]+=units[i][j];
    }
    overall+=total[i];
}

// (a) Annual Consumption
printf("\nAnnual Consumption:\n");
for(i=0;i<n;i++)
    printf("Household %d = %.2f Units\n",i+1,total[i]);

// (b) Maximum and Minimum Consumption
for(i=1;i<n;i++)
{
    if(total[i]>total[maxHouse])
        maxHouse=i;
    if(total[i]<total[minHouse])
        minHouse=i;
}

printf("\nMaximum Consumption: Household %d (%.2f Units)\n",
        maxHouse+1,total[maxHouse]);
printf("Minimum Consumption: Household %d (%.2f Units)\n",
        minHouse+1,total[minHouse]);

// (c) Month with Highest Overall Consumption
maxTotal=0;
for(j=0;j<12;j++)
{
    monthTotal=0;
    for(i=0;i<n;i++)

```

```

        monthTotal+=units[i][j];
    if(monthTotal>maxTotal)
    {
        maxTotal=monthTotal;
        maxMonth=j;
    }
}
printf("\nMonth with Highest Consumption = %d\n",maxMonth+1);
// (d) Increasing Consumption for at least 3 Months
printf("\nHouseholds with Increasing Consumption:\n");
int found=0;
for(i=0;i<n;i++)
{
    int count=1;
    for(j=1;j<12;j++)
    {
        if(units[i][j]>units[i][j-1])
            count++;
        else
            count=1;
        if(count>=3)
        {
            printf("Household %d\n",i+1);
            found=1;
            break;
        }
    }
}
if(found==0)
    printf("None\n");
// (e) Above Annual Average
float average=overall/n;

```

```

printf("\nHouseholds Above Annual Average:\n");

found=0;

for(i=0;i<n;i++)
{
    if(total[i]>average)
    {
        printf("Household %d\n",i+1);
        found=1;
    }
}

if(found==0)

    printf("None\n");

return 0;
}

```

Output

```

1 #include<stdio.h>
2 int main()
3 {
4     int n,i,j;
5     float units[20][12];
6     float total[20]=0;
7     float overall=0;
8     int month=0, maxHouse=0;
9     int maxMonth=0;
10    float maxTotal, minTotal=0;
11    printf("Enter number of households: ");
12    scanf("%d",&n);
13    printf("Enter electricity consumption for 12 months:\n");
14    for(i=0;i<n;i++)
15    {
16        printf("Household %d:\n",i+1);
17        for(j=0;j<12;j++)
18        {
19            scanf("%f",&units[i][j]);
20            total[i]+=units[i][j];
21        }
22        overall+=total[i];
23    }
24    // (a) Annual Consumption
25    printf("Annual Consumption:\n");
26    for(i=0;i<n;i++)
27    {
28        printf("Household %d = %.2f Units\n",i+1,total[i]);
29        // (b) Maximum and Minimum Consumption
30        for(k=0;k<n;k++)
31        {
32            if(total[i]>total[maxHouse])
33                maxHouse=i;
34            if(total[i]<total[minHouse])
35                minHouse=i;
36        }
37    }
38    printf("Maximum Consumption: Household %d (%.2f Units)\n",
39           maxHouse+1,total[maxHouse]);
40    printf("Minimum Consumption: Household %d (%.2f Units)\n",
41           minHouse+1,total[minHouse]);
42    // (c) Month with Highest Overall Consumption
43    maxTotal=0;
44    for(j=0;j<12;j++)

```

```

Enter number of households: 2
Enter electricity consumption for 12 months:

Household 1:
145
189
496
4852
756
12
1661
145
174
741
896
852

Household 2:
145
256
4502
145
856
3965
485
142
1452
12
1
20

Annual Consumption:
Household 1 = 18137.90 Units
Household 2 = 8683.90 Units

Maximum Consumption: Household 2 (18682.80 Units)
Minimum Consumption: Household 1 (18137.90 Units)

Month with Highest Consumption = 6.

Households with Increasing Consumption:

```

Question 11: Recursive Number Pattern Analyzer

Aim

To develop a C program using recursion to analyze the digits of a number.

Program

```
#include <stdio.h>
```



```

int sumDigits(int n)
{
    if(n == 0)
        return 0;
    return (n % 10) + sumDigits(n / 10);
}

int productDigits(int n)
{
    if(n == 0)
        return 1;
    return (n % 10) * productDigits(n / 10);
}

int reverseNumber(int n, int rev)
{
    if(n == 0)
        return rev;
    return reverseNumber(n / 10, rev * 10 + n % 10);
}

int largestDigit(int n)
{
    if(n < 10)
        return n;
    int max = largestDigit(n / 10);
    if(n % 10 > max)
        return n % 10;
    else
        return max;
}

int digitCount(int n)
{
    if(n == 0)
        return 0;

```

```

        return 1 + digitCount(n / 10);
    }

int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    // (a) Sum of digits
    printf("\nSum of Digits = %d", sumDigits(num));

    // (b) Product of digits
    printf("\nProduct of Digits = %d", productDigits(num));

    // (c) Reverse of number
    printf("\nReverse = %d", reverseNumber(num, 0));

    // (d) Largest digit
    printf("\nLargest Digit = %d", largestDigit(num));

    // (e) Number of digits
    printf("\nNumber of Digits = %d", digitCount(num));

    return 0;
}

```

Output

The screenshot shows a C++ IDE with the following code and output:

```

main.cpp
1 #include <stdio.h>
2 int sumDigits(int n)
3 {
4     if(n == 0)
5         return 0;
6     return (n % 10) + sumDigits(n / 10);
7 }
8 int productDigits(int n)
9 {
10    if(n == 0)
11        return 1;
12    return (n % 10) * productDigits(n / 10);
13 }
14 int reverseNumber(int n, int rev)
15 {
16    if(n == 0)
17        return rev;
18    return reverseNumber(n / 10, rev * 10 + n % 10);
19 }
20 int largestDigit(int n)
21 {
22    if(n < 10)
23        return n;
24    int max = largestDigit(n / 10);
25    if(n % 10 > max)
26        return n % 10;
27    else
28        return max;
29 }
30 int digitCount(int n)
31 {
32    if(n == 0)
33        return 0;
34    return 1 + digitCount(n / 10);

```

Output:

```

Enter a number: 14562
Sum of Digits = 18
Product of Digits = 240
Reverse = 24561
Largest Digit = 6
Number of Digits = 5
--- Code Execution Successful ---

```

12. Employee Salary and Increment Analyzer

Aim

To develop a C program using structures to store employee details, calculate gross salary, apply increment based on experience, identify the highest-paid employee in each department, calculate the average salary, and display employees whose revised salary is above the organization average.

Program Code

```
#include <stdio.h>
#include <string.h>
struct Employee
{
    int id;
    char name[30];
    char dept[30];
    float basic, gross, revised;
    int exp;
};
int main()
{
    struct Employee e[100];
    int n, i, j;
    printf("Enter number of employees: ");
    scanf("%d", &n);
    for(i=0;i<n;i++)
    {
        printf("\nEmployee %d\n", i+1);
        printf("ID: ");
        scanf("%d",&e[i].id);
        printf("Name: ");
        scanf("%s",e[i].name);
        printf("Department: ");
        scanf("%s",e[i].dept);
        printf("Basic Salary: ");
```

```

scanf("%f",&e[i].basic);
printf("Experience (years): ");
scanf("%d",&e[i].exp);
// Gross Salary
e[i].gross = e[i].basic + (0.20 * e[i].basic);
// Increment based on experience
if(e[i].exp >= 10)
    e[i].revised = e[i].gross + (0.15 * e[i].gross);
else if(e[i].exp >= 5)
    e[i].revised = e[i].gross + (0.10 * e[i].gross);
else
    e[i].revised = e[i].gross + (0.05 * e[i].gross);
}
float total = 0;
printf("\nEmployee Salary Details\n");
printf("-----\n");
printf("ID\tName\tDept\tGross\tRevised\n");
for(i=0;i<n;i++)
{
    printf("%d\t%s\t%s\t%.2f\t%.2f\n",
        e[i].id,e[i].name,e[i].dept,e[i].gross,e[i].revised);
    total += e[i].revised;
}
float avg = total/n;
printf("\nAverage Revised Salary = %.2f\n",avg);
printf("\nEmployees Above Average Salary\n");
for(i=0;i<n;i++)
{
    if(e[i].revised>avg)
        printf("%s (%.2f)\n",e[i].name,e[i].revised);
}
// Highest paid in each department

```

```

printf("\nHighest Paid Employee in Each Department\n");
for(i=0;i<n;i++)
{
    int printed=0;
    for(j=0;j<i;j++)
    {
        if(strcmp(e[i].dept,e[j].dept)==0)
        {
            printed=1;
            break;
        }
    }
    if(!printed)
    {
        int max=i;
        for(j=i+1;j<n;j++)
        {
            if(strcmp(e[i].dept,e[j].dept)==0 &&
                e[j].revised>e[max].revised)
                max=j;
        }
        printf("%s -> %s (%.2f)\n",
            e[max].dept,e[max].name,e[max].revised);
    }
}
return 0;
}

```

Output

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 struct Employee
4 {
5     int id;
6     char name[30];
7     char dept[30];
8     float basic, gross, revised;
9     int exp;
10 };
11 int main()
12 {
13     struct Employee e[100];
14     int n, i, j;
15     printf("Enter number of employees: ");
16     scanf("%d", &n);
17     for(i=0; i<n; i++)
18     {
19         printf("\nEnter Employee %d\n", i+1);
20         printf("ID: ");
21         scanf("%d", &e[i].id);
22         printf("Name: ");
23         scanf("%s", e[i].name);
24         printf("Department: ");
25         scanf("%s", e[i].dept);
26         printf("Basic Salary: ");
27         scanf("%f", &e[i].basic);
28         printf("Experience (years): ");
29         scanf("%d", &e[i].exp);
30         // Gross Salary
31         e[i].gross = e[i].basic + (0.20 * e[i].basic);
32         // Increment based on experience
33         if(e[i].exp >= 10)
34             e[i].revised = e[i].gross + (0.15 * e[i].gross);
35     }
36     // Output
37     printf("\nEmployee Details:\n");
38     printf("ID Name Dept Gross Revised\n");
39     for(i=0; i<n; i++)
40     {
41         printf("%d %s %s %f %f\n", e[i].id, e[i].name, e[i].dept, e[i].gross, e[i].revised);
42     }
43     // Average Revised Salary
44     float avg = 0.0;
45     for(i=0; i<n; i++)
46         avg += e[i].revised;
47     avg /= n;
48     printf("Average Revised Salary = %f\n", avg);
49     // Employees Above Average Salary
50     printf("Employees Above Average Salary:\n");
51     for(i=0; i<n; i++)
52     {
53         if(e[i].revised > avg)
54             printf("%d %s %s %f %f\n", e[i].id, e[i].name, e[i].dept, e[i].gross, e[i].revised);
55     }
56     // Highest Paid Employee in Each Department
57     printf("Highest Paid Employee in Each Department:\n");
58     for(i=0; i<n; i++)
59     {
60         if(e[i].dept[0] != '\0')
61             printf("%s -> %s (%f)\n", e[i].dept, e[i].name, e[i].revised);
62     }
63 }
```

Output

```
Enter number of employees: 1
Employee 1
ID: 192425194
Name: Shaik Mubarak
Department: Basic Salary: CSE:52045220
Experience (years):
Employee Salary Details:
-----
ID Name Dept Gross Revised
192425194 Shaik Mubarak 0.00 0.00
Average Revised Salary = 0.00
Employees Above Average Salary
Highest Paid Employee in Each Department
Mubarak -> Shaik (0.00)
--- Code Execution Successful ---
```

13. Matrix Transformation and Equality Analyzer

Aim

To develop a C program to compute the transpose of two matrices, rotate them by 90° clockwise, determine whether one matrix can be transformed into the other through rotation, count the number of different elements, and display the required transformation.

Program Code

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j;
```

```
    int A[10][10],B[10][10];
```

```
    int T[10][10],R[10][10];
```

```
    printf("Enter order of square matrix: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter Matrix A:\n");
```

```
    for(i=0;i<n;i++)
```

```
        for(j=0;j<n;j++)
```

```
            scanf("%d",&A[i][j]);
```

```
    printf("Enter Matrix B:\n");
```

```

for(i=0;i<n;i++)
    for(j=0;j<n;j++)
        scanf("%d",&B[i][j]);
printf("\nTranspose of Matrix A\n");
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        T[i][j]=A[j][i];
        printf("%d ",T[i][j]);
    }
    printf("\n");
}
printf("\nTranspose of Matrix B\n");
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
        printf("%d ",B[j][i]);
    printf("\n");
}
//90 degree rotation of A
for(i=0;i<n;i++)
    for(j=0;j<n;j++)
        R[j][n-1-i]=A[i][j];
printf("\n90 Degree Rotation of Matrix A\n");
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
        printf("%d ",R[i][j]);
    printf("\n");
}
int same=1;

```

```

for(i=0;i<n;i++)
    for(j=0;j<n;j++)
        if(R[i][j]!=B[i][j])
            same=0;
if(same)
    printf("\nMatrix A becomes Matrix B after 90 degree rotation.\n");
else
    printf("\nMatrix A cannot be transformed into Matrix B by 90 degree rotation.\n");
int diff=0;
for(i=0;i<n;i++)
    for(j=0;j<n;j++)
        if(A[i][j]!=B[i][j])
            diff++;
printf("Different Positions = %d\n",diff);
return 0;
}

```

Output

The screenshot shows a C++ IDE with the following code in the editor:

```

1 #include <stdio.h>
2 int main()
3 {
4     int n,i,j;
5     int A[10][10],B[10][10];
6     int T[10][10],R[10][10];
7     printf("Enter order of square matrix: ");
8     scanf("%d",&n);
9     printf("Enter Matrix A:\n");
10    for(i=0;i<n;i++)
11        for(j=0;j<n;j++)
12            scanf("%d",&A[i][j]);
13    printf("Enter Matrix B:\n");
14    for(i=0;i<n;i++)
15        for(j=0;j<n;j++)
16            scanf("%d",&B[i][j]);
17    printf("\nTranspose of Matrix A:\n");
18    for(i=0;i<n;i++)
19    {
20        for(j=0;j<n;j++)
21        {
22            T[i][j]=A[j][i];
23            printf("%d ",T[i][j]);
24        }
25        printf("\n");
26    }
27    printf("\nTranspose of Matrix B:\n");
28    for(i=0;i<n;i++)
29    {
30        for(j=0;j<n;j++)
31            printf("%d ",B[j][i]);
32        printf("\n");
33    }
34 }

```

The output window shows the following results:

```

Enter order of square matrix: 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
5 6
7 8
Transpose of Matrix A
1 3
2 4
Transpose of Matrix B
5 7
6 8
90 Degree Rotation of Matrix A
3 1
4 2
Matrix A cannot be transformed into Matrix B by 90 degree rotation.
Different Positions = 4
--- Code Execution Successful ---

```


14. File-Based Inventory Analyzer

Aim

To develop a C program using file handling to store product records, search products by Product ID, calculate total inventory value, identify low-stock products, and generate a report.

Program Code

```
#include <stdio.h>

struct Product
{
    int id;
    char name[30];
    int qty;
    float price;
};

int main()
{
    FILE *fp;
    struct Product p;
    int n,i,search,reorder;
    float total=0;
    fp=fopen("inventory.txt","w");
    printf("Enter number of products: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nProduct %d\n",i+1);
        printf("ID: ");
        scanf("%d",&p.id);
        printf("Name: ");
        scanf("%s",p.name);
        printf("Quantity: ");
        scanf("%d",&p.qty);
        printf("Price: ");
```

```
        scanf("%f",&p.price);
        fwrite(&p,sizeof(p),1,fp);
    }
    fclose(fp);
    fp=fopen("inventory.txt","r");
    printf("\nEnter Product ID to search: ");
    scanf("%d",&search);
    while(fread(&p,sizeof(p),1,fp))
    {
        if(p.id==search)
        {
            printf("\nProduct Found\n");
            printf("%d %s %d %.2f\n",
                p.id,p.name,p.qty,p.price);
        }
    }
    rewind(fp);
    printf("\nEnter Reorder Level: ");
    scanf("%d",&reorder);
    printf("\nLow Stock Products\n");
    while(fread(&p,sizeof(p),1,fp))
    {
        total += p.qty*p.price;
        if(p.qty<reorder)
            printf("%s\n",p.name);
    }
    printf("\nTotal Inventory Value = %.2f\n",total);
    fclose(fp);
    return 0;
}
```

Output

Output	Clear
Enter number of products: 2	
Product 1	
ID: 101	
Name: pen	
Quantity: 50	
Price: 10	
Segmentation fault	

15. Multi-Level Data Search and Ranking System

Aim

To develop a C program using structures to search student records by Register Number and Department, generate CGPA ranking without modifying original records, identify placement-eligible students, and display the top three eligible students from each department.

Program Code

```
#include <stdio.h>
#include <string.h>
struct Student
{
    int regno;
    char name[30];
    char dept[20];
    float cgpa;
    int backlog;
};
int main()
{
    struct Student s[50], temp;
    int n,i,j,reg;
    char dept[20];
    printf("Enter number of students: ");
```

```

scanf("%d",&n);
for(i=0;i<n;i++)
{
    printf("\nStudent %d\n",i+1);
    printf("Register Number: ");
    scanf("%d",&s[i].regno);
    printf("Name: ");
    scanf("%s",s[i].name);
    printf("Department: ");
    scanf("%s",s[i].dept);
    printf("CGPA: ");
    scanf("%f",&s[i].cgpa);
    printf("Backlogs: ");
    scanf("%d",&s[i].backlog);
}
printf("\nEnter Register Number to Search: ");
scanf("%d",&reg);
for(i=0;i<n;i++)
{
    if(s[i].regno==reg)
    {
        printf("\nRecord Found\n");
        printf("%d %s %s %.2f %d\n",s[i].regno,s[i].name,s[i].dept,s[i].cgpa,s[i].backlog);
    }
}
printf("\nEnter Department: ");
scanf("%s",dept);
printf("\nStudents in %s Department\n",dept);
for(i=0;i<n;i++)
{
    if(strcmp(s[i].dept,dept)==0)
        printf("%s %.2f\n",s[i].name,s[i].cgpa);
}

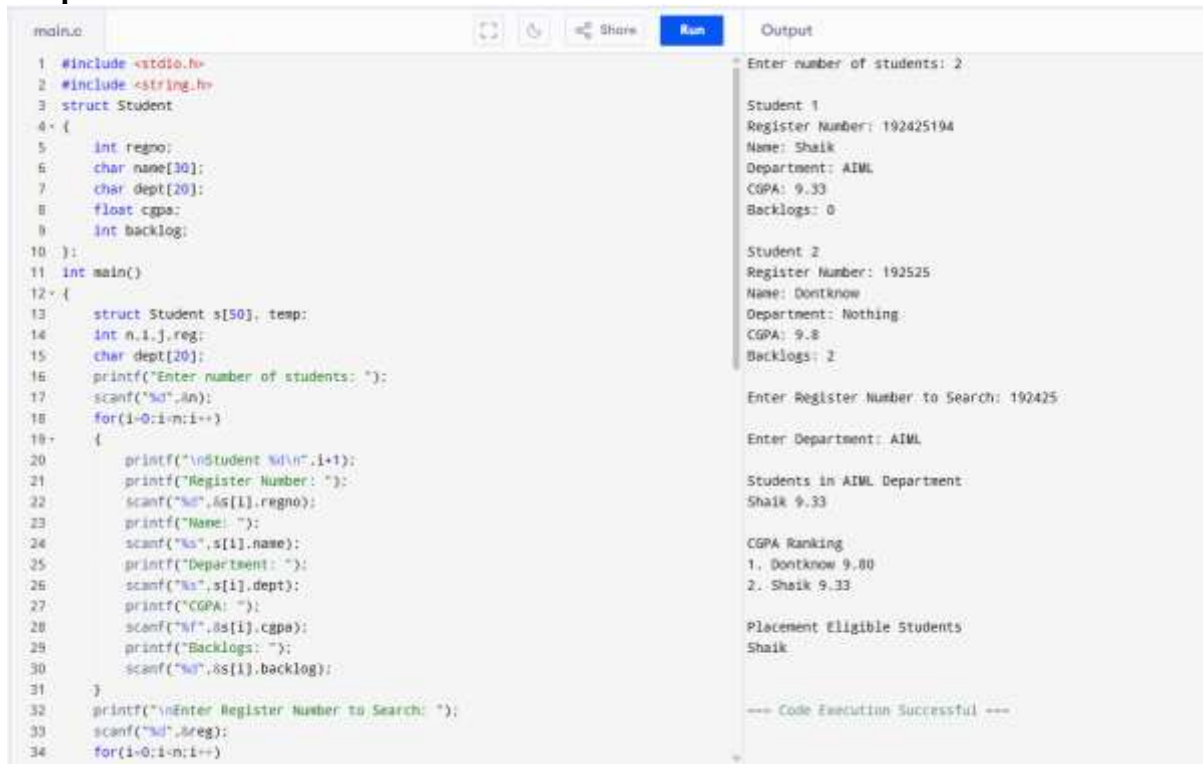
```

```

}
struct Student rank[50];
for(i=0;i<n;i++)
    rank[i]=s[i];
for(i=0;i<n-1;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(rank[j].cgpa>rank[i].cgpa)
        {
            temp=rank[i];
            rank[i]=rank[j];
            rank[j]=temp;
        }
    }
}
printf("\nCGPA Ranking\n");
for(i=0;i<n;i++)
    printf("%d. %s %.2f\n",i+1,rank[i].name,rank[i].cgpa);
printf("\nPlacement Eligible Students\n");
for(i=0;i<n;i++)
{
    if(s[i].cgpa>=7.5 && s[i].backlog==0)
        printf("%s\n",s[i].name);
}
return 0;
}

```

Output



The screenshot shows a C program being executed in an IDE. The code defines a 'Student' struct with fields for register number, name, department, CGPA, and backlogs. It then prompts the user to enter the number of students, followed by details for each student. After inputting data for two students, the program displays a summary for the AIML department, showing CGPA rankings and placement eligibility.

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 struct Student
4 {
5     int regno;
6     char name[30];
7     char dept[20];
8     float cgpa;
9     int backlog;
10 };
11 int main()
12 {
13     struct Student s[50], temp;
14     int n,i,j,reg;
15     char dept[20];
16     printf("Enter number of students: ");
17     scanf("%d",&n);
18     for(i=0;i<n;i++)
19     {
20         printf("\nStudent %d\n",i+1);
21         printf("Register Number: ");
22         scanf("%d",&s[i].regno);
23         printf("Name: ");
24         scanf("%s",s[i].name);
25         printf("Department: ");
26         scanf("%s",s[i].dept);
27         printf("CGPA: ");
28         scanf("%f",&s[i].cgpa);
29         printf("Backlogs: ");
30         scanf("%d",&s[i].backlog);
31     }
32     printf("\nEnter Register Number to Search: ");
33     scanf("%d",&reg);
34     for(i=0;i<n;i++)
```

Output

```
Enter number of students: 2

Student 1
Register Number: 192425194
Name: Shaik
Department: AIML
CGPA: 9.33
Backlogs: 0

Student 2
Register Number: 192525
Name: Dontknow
Department: Nothing
CGPA: 9.8
Backlogs: 2

Enter Register Number to Search: 192425

Enter Department: AIML

Students in AIML Department
Shaik 9.33

CGPA Ranking
1. Dontknow 9.80
2. Shaik 9.33

Placement Eligible Students
Shaik

--- Code Execution Successful ---
```

17. Votes Received by Candidates

Aim

To develop a C program to sort candidates based on votes, identify the winner and runner-up, and detect election ties.

Program Code

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j,temp;
```

```
    printf("Enter number of candidates: ");
```

```
    scanf("%d",&n);
```

```
    int votes[20];
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        printf("Votes of Candidate %d: ",i+1);
```

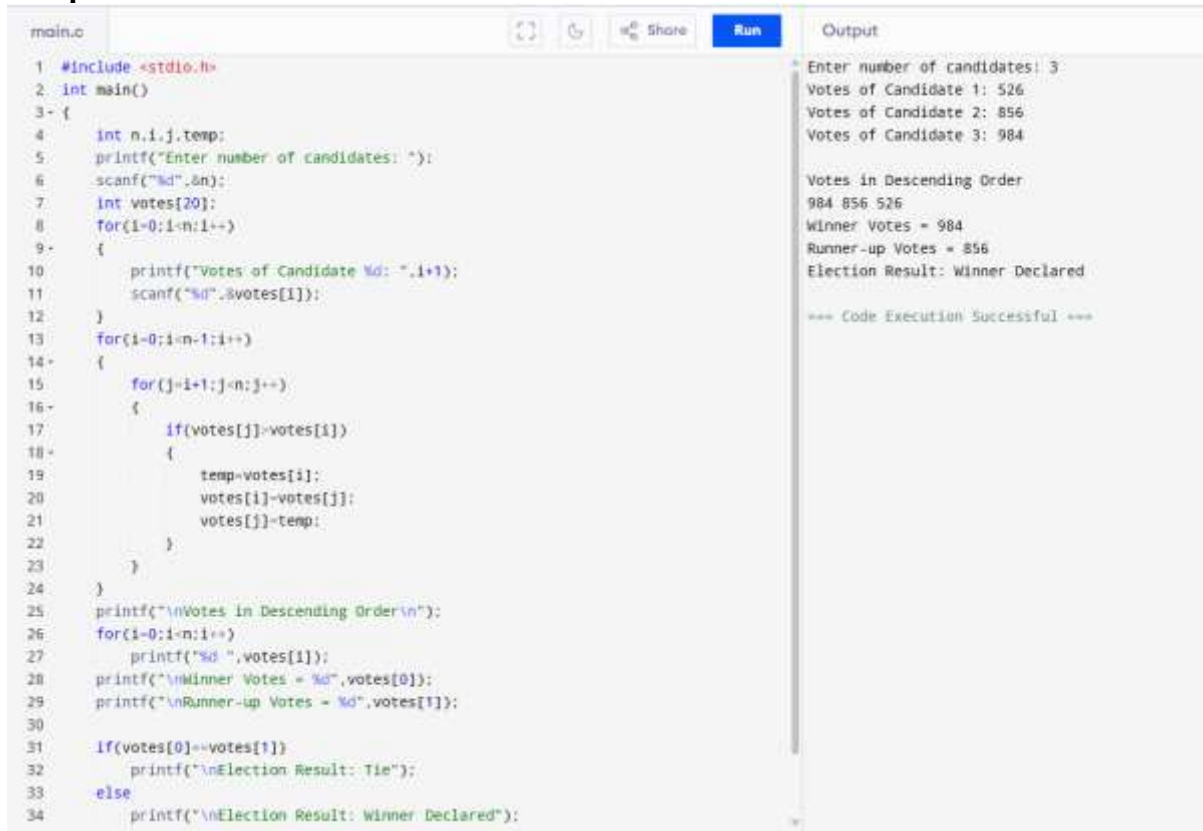
```
        scanf("%d",&votes[i]);
```

```
    }
```

```
for(i=0;i<n-1;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(votes[j]>votes[i])
        {
            temp=votes[i];
            votes[i]=votes[j];
            votes[j]=temp;
        }
    }
}
printf("\nVotes in Descending Order\n");
for(i=0;i<n;i++)
    printf("%d ",votes[i]);
printf("\nWinner Votes = %d",votes[0]);
printf("\nRunner-up Votes = %d",votes[1]);

if(votes[0]==votes[1])
    printf("\nElection Result: Tie");
else
    printf("\nElection Result: Winner Declared");
return 0;
}
```

Output



The screenshot shows a C program in a code editor with a 'Run' button. The program calculates the winner of an election based on votes from 3 candidates. The output window shows the following results:

```
main.c
1 #include <stdio.h>
2 int main()
3 {
4     int n,i,j,temp;
5     printf("Enter number of candidates: ");
6     scanf("%d",&n);
7     int votes[20];
8     for(i=0;i<n;i++)
9     {
10         printf("Votes of Candidate %d: ",i+1);
11         scanf("%d",&votes[i]);
12     }
13     for(i=0;i<n-1;i++)
14     {
15         for(j=i+1;j<n;j++)
16         {
17             if(votes[j]>votes[i])
18             {
19                 temp=votes[i];
20                 votes[i]=votes[j];
21                 votes[j]=temp;
22             }
23         }
24     }
25     printf("\nVotes in Descending Order\n");
26     for(i=0;i<n;i++)
27         printf("%d ",votes[i]);
28     printf("\nWinner Votes = %d",votes[0]);
29     printf("\nRunner-up Votes = %d",votes[1]);
30
31     if(votes[0]==votes[1])
32         printf("\nElection Result: Tie");
33     else
34         printf("\nElection Result: Winner Declared");
```

Output

```
Enter number of candidates: 3
Votes of Candidate 1: 526
Votes of Candidate 2: 856
Votes of Candidate 3: 984

Votes in Descending Order
984 856 526
Winner Votes = 984
Runner-up Votes = 856
Election Result: Winner Declared

=== Code Execution Successful ===
```

18. Matrix Classification

Aim

To develop a C program to determine whether a square matrix is Symmetric, Upper Triangular, Lower Triangular, Diagonal, or Identity Matrix.

Program Code

```
#include <stdio.h>

int main()
{
    int n,i,j;
    int a[10][10];
    int sym=1,upper=1,lower=1,diag=1,identity=1;
    printf("Enter order of matrix: ");
    scanf("%d",&n);
    printf("Enter matrix elements:\n");
    for(i=0;i<n;i++)
```



```

    for(j=0;j<n;j++)
        scanf("%d",&a[i][j]);
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        if(a[i][j]!=a[j][i])
            sym=0;
        if(i>j && a[i][j]!=0)
            upper=0;
        if(i<j && a[i][j]!=0)
            lower=0;
        if(i!=j && a[i][j]!=0)
            diag=0;
        if(i==j)
        {
            if(a[i][j]!=1)
                identity=0;
        }
        else
        {
            if(a[i][j]!=0)
                identity=0;
        }
    }
}
printf("\nApplicable Classifications\n");
if(sym)
    printf("Symmetric Matrix\n");
if(upper)
    printf("Upper Triangular Matrix\n");
if(lower)

```

```

        printf("Lower Triangular Matrix\n");
    if(diag)
        printf("Diagonal Matrix\n");
    if(identity)
        printf("Identity Matrix\n");
    return 0;
}

```

Output

The screenshot shows a C program in a code editor with a 'Run' button. The code defines a function to check if a matrix is symmetric, upper triangular, lower triangular, diagonal, or identity. The output window shows the program's execution with the following text:

```

Enter order of matrix: 3
Enter matrix elements:
1 5 6
7 5 9
1 6 8

Applicable Classifications

--- Code Execution Successful ---

```

19. Search Method Analyzer

Aim

To develop a C program to determine whether an array is sorted. If sorted, perform Binary Search; otherwise, perform Linear Search. Also display the search method used and the number of comparisons.

Program Code

```

#include <stdio.h>

int main()
{
    int n, a[100], i, key;

```

```

int sorted = 1;
int comparisons = 0;
printf("Enter number of elements: ");
scanf("%d", &n);
printf("Enter array elements:\n");
for(i = 0; i < n; i++)
    scanf("%d", &a[i]);
printf("Enter element to search: ");
scanf("%d", &key);
for(i = 0; i < n - 1; i++)
{
    if(a[i] > a[i + 1])
    {
        sorted = 0;
        break;
    }
}
if(sorted)
{
    printf("\nSearch Method: Binary Search\n");
    int low = 0, high = n - 1, mid;
    int found = 0;
    while(low <= high)
    {
        comparisons++;
        mid = (low + high) / 2;
        if(a[mid] == key)
        {
            printf("Element Found at Position %d\n", mid + 1);
            found = 1;
            break;
        }
    }
}

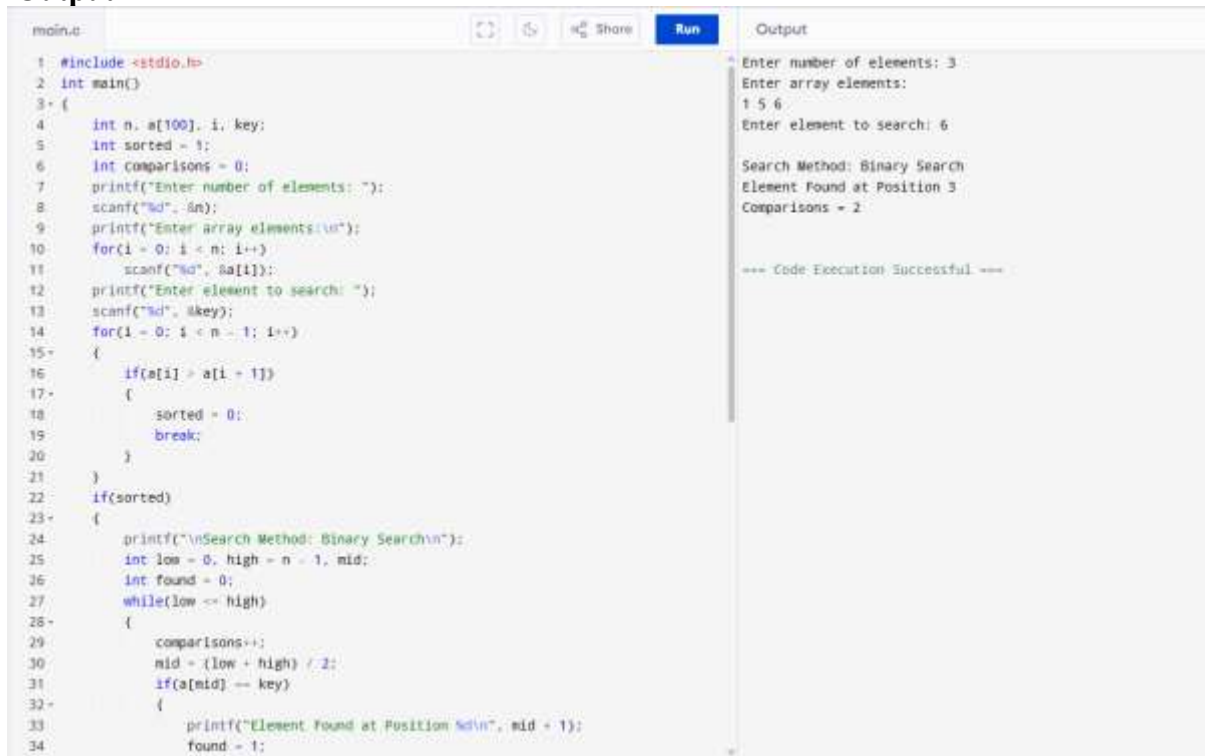
```

```

        else if(a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    if(!found)
        printf("Element Not Found\n");
}
else
{
    printf("\nSearch Method: Linear Search\n");
    int found = 0;
    for(i = 0; i < n; i++)
    {
        comparisons++;
        if(a[i] == key)
        {
            printf("Element Found at Position %d\n", i + 1);
            found = 1;
            break;
        }
    }
    if(!found)
        printf("Element Not Found\n");
}
printf("Comparisons = %d\n", comparisons);
return 0;
}

```

Output



The screenshot shows a C program in a code editor with a 'Run' button and an 'Output' pane. The code implements a binary search algorithm. The output pane shows the user's input and the program's response.

```
main.c
1 #include <stdio.h>
2 int main()
3 {
4     int n, a[100], i, key;
5     int sorted = 1;
6     int comparisons = 0;
7     printf("Enter number of elements: ");
8     scanf("%d", &n);
9     printf("Enter array elements:\n");
10    for(i = 0; i < n; i++)
11        scanf("%d", &a[i]);
12    printf("Enter element to search: ");
13    scanf("%d", &key);
14    for(i = 0; i < n - 1; i++)
15    {
16        if(a[i] > a[i + 1])
17        {
18            sorted = 0;
19            break;
20        }
21    }
22    if(sorted)
23    {
24        printf("\nSearch Method: Binary Search\n");
25        int low = 0, high = n - 1, mid;
26        int found = 0;
27        while(low <= high)
28        {
29            comparisons++;
30            mid = (low + high) / 2;
31            if(a[mid] == key)
32            {
33                printf("Element Found at Position %d\n", mid + 1);
34                found = 1;
35            }
36        }
37    }
38}
```

Output

```
Enter number of elements: 3
Enter array elements:
1 5 6
Enter element to search: 6

Search Method: Binary Search
Element Found at Position 3
Comparisons = 2

=== Code Execution Successful ===
```

20. Student Marks Report

Aim

To develop a C program using a two-dimensional array to generate a student marks report including student topper, subject topper, class average, subject-wise average, students scoring above overall average, and rank list.

Program Code

```
#include <stdio.h>

int main()
{
    int n, m;
    int marks[20][20];
    int total[20];
    float classAvg = 0;
    int i, j;
    printf("Enter number of students: ");
    scanf("%d", &n);
    printf("Enter number of subjects: ");
```

```

scanf("%d", &m);
for(i = 0; i < n; i++)
{
    total[i] = 0;
    printf("\nEnter marks of Student %d:\n", i + 1);
    for(j = 0; j < m; j++)
    {
        scanf("%d", &marks[i][j]);
        total[i] += marks[i][j];
    }
    classAvg += total[i];
}
classAvg = classAvg / n;
int topper = 0;
for(i = 1; i < n; i++)
{
    if(total[i] > total[topper])
        topper = i;
}
printf("\nStudent Topper : Student %d (%d Marks)\n",
    topper + 1, total[topper]);
printf("\nSubject Toppers\n");
for(j = 0; j < m; j++)
{
    int max = 0;
    for(i = 1; i < n; i++)
    {
        if(marks[i][j] > marks[max][j])
            max = i;
    }
    printf("Subject %d : Student %d (%d)\n",
        j + 1, max + 1, marks[max][j]);
}

```

```

}
printf("\nClass Average = %.2f\n", classAvg);
printf("\nSubject-wise Average\n");
for(j = 0; j < m; j++)
{
    float sum = 0;
    for(i = 0; i < n; i++)
        sum += marks[i][j];
    printf("Subject %d = %.2f\n",
        j + 1, sum / n);
}
printf("\nStudents Above Overall Average\n");
for(i = 0; i < n; i++)
{
    if(total[i] > classAvg)
        printf("Student %d\n", i + 1);
}
printf("\nRank List\n");
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(total[j] > total[i])
        {
            int t = total[i];
            total[i] = total[j];
            total[j] = t;
        }
    }
}
for(i = 0; i < n; i++)
    printf("Rank %d : %d Marks\n", i + 1, total[i]);

```

```

return 0;
}

```

Output

The screenshot shows a C program in a code editor with a 'Run' button. The code calculates the class average and identifies the top student and subject toppers based on input marks for 1 student and 3 subjects. The output on the right matches the program's execution.

```

main.c
1 #include <stdio.h>
2 int main()
3 {
4     int n, m;
5     int marks[20][20];
6     int total[20];
7     float classAvg = 0;
8     int i, j;
9     printf("Enter number of students: ");
10    scanf("%d", &n);
11    printf("Enter number of subjects: ");
12    scanf("%d", &m);
13    for(i = 0; i < n; i++)
14    {
15        total[i] = 0;
16        printf("\nEnter marks of Student %d:\n", i + 1);
17        for(j = 0; j < m; j++)
18        {
19            scanf("%d", &marks[i][j]);
20            total[i] += marks[i][j];
21        }
22        classAvg += total[i];
23    }
24    classAvg = classAvg / n;
25    int topper = 0;
26    for(i = 1; i < n; i++)
27    {
28        if(total[i] > total[topper])
29            topper = i;
30    }
31    printf("\nStudent Topper : Student %d (%d Marks)\n",
32           topper + 1, total[topper]);
33    printf("\nSubject Toppers\n");
34    for(j = 0; j < m; j++)

```

Output

```

Enter number of students: 1
Enter number of subjects: 3

Enter marks of Student 1:
95 98 95

Student Topper : Student 1 (288 Marks)

Subject Toppers
Subject 1 : Student 1 (95)
Subject 2 : Student 1 (98)
Subject 3 : Student 1 (95)

Class Average = 288.00

Subject-wise Average
Subject 1 = 95.00
Subject 2 = 98.00
Subject 3 = 95.00

Students Above Overall Average

Rank List
Rank 1 : 288 Marks

=== Code Execution Successful ===

```

21(a). Merge Three Sorted Arrays

Aim

To develop a C program to merge three sorted arrays into a single sorted array while removing duplicate elements without using any library sorting function.

Program Code

```

#include <stdio.h>

int main()
{
    int a[50], b[50], c[50], temp[150], result[150];
    int n1, n2, n3;
    int i, j, k = 0, m = 0;
    printf("Enter size of first array: ");

```



```

scanf("%d", &n1);
printf("Enter elements of first sorted array:\n");
for(i = 0; i < n1; i++)
    scanf("%d", &a[i]);
printf("Enter size of second array: ");
scanf("%d", &n2);
printf("Enter elements of second sorted array:\n");
for(i = 0; i < n2; i++)
    scanf("%d", &b[i]);
printf("Enter size of third array: ");
scanf("%d", &n3);
printf("Enter elements of third sorted array:\n");
for(i = 0; i < n3; i++)
    scanf("%d", &c[i]);
for(i = 0; i < n1; i++)
    temp[k++] = a[i];
for(i = 0; i < n2; i++)
    temp[k++] = b[i];
for(i = 0; i < n3; i++)
    temp[k++] = c[i];
for(i = 0; i < k - 1; i++)
{
    for(j = 0; j < k - i - 1; j++)
    {
        if(temp[j] > temp[j + 1])
        {
            int t = temp[j];
            temp[j] = temp[j + 1];
            temp[j + 1] = t;
        }
    }
}
}

```

```

result[m++] = temp[0];
for(i = 1; i < k; i++)
{
    if(temp[i] != temp[i - 1])
        result[m++] = temp[i];
}
printf("\nMerged Sorted Array without Duplicates:\n");
for(i = 0; i < m; i++)
    printf("%d ", result[i]);
return 0;
}

```

Output

The screenshot shows a C program in a code editor with the following code:

```

1 #include <stdio.h>
2 int main()
3 {
4     int a[50], b[50], c[50], temp[150], result[150];
5     int n1, n2, n3;
6     int i, j, k = 0, m = 0;
7     printf("Enter size of first array: ");
8     scanf("%d", &n1);
9     printf("Enter elements of first sorted array:\n");
10    for(i = 0; i < n1; i++)
11        scanf("%d", &a[i]);
12    printf("Enter size of second array: ");
13    scanf("%d", &n2);
14    printf("Enter elements of second sorted array:\n");
15    for(i = 0; i < n2; i++)
16        scanf("%d", &b[i]);
17    printf("Enter size of third array: ");
18    scanf("%d", &n3);
19    printf("Enter elements of third sorted array:\n");
20    for(i = 0; i < n3; i++)
21        scanf("%d", &c[i]);
22    for(i = 0; i < n1; i++)
23        temp[k++] = a[i];
24    for(i = 0; i < n2; i++)
25        temp[k++] = b[i];
26    for(i = 0; i < n3; i++)
27        temp[k++] = c[i];
28    for(i = 0; i < k - 1; i++)
29    {
30        for(j = 0; j = k - i - 1; j++)
31        {
32            if(temp[j] > temp[j + 1])
33            {
34                int t = temp[j];

```

The output on the right shows the following sequence of inputs and results:

```

Enter size of first array: 5
Enter elements of first sorted array:
1 5 9 15 58
Enter size of second array: 5
Enter elements of second sorted array:
1 2 3 4 5
Enter size of third array: 3
Enter elements of third sorted array:
1 2 6

Merged Sorted Array without Duplicates:
1 2 3 4 5 6 9 15 58

```

At the bottom, it says "Code Execution Successful".

21(b). Matrix Rotation (90°, 180°, and 270°)

Aim

To develop a C program to rotate a square matrix by **90°, 180°, and 270° clockwise** and determine whether any rotated matrix is identical to the original matrix.

Program Code

```
#include <stdio.h>
```

```

int main()
{
    int n, i, j;
    int a[10][10];
    int r90[10][10], r180[10][10], r270[10][10];
    int same = 1;
    printf("Enter order of matrix: ");
    scanf("%d", &n);
    printf("Enter matrix elements:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            r90[j][n - 1 - i] = a[i][j];
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            r180[j][n - 1 - i] = r90[i][j];
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            r270[j][n - 1 - i] = r180[i][j];
    printf("\n90 Degree Rotation\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d ", r90[i][j]);
        printf("\n");
    }
    printf("\n180 Degree Rotation\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)

```

```

        printf("%d ", r180[i][j]);
    printf("\n");
}
printf("\n270 Degree Rotation\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
        printf("%d ", r270[i][j]);
    printf("\n");
}
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        if(a[i][j] != r90[i][j])
        {
            same = 0;
            break;
        }
    }
}
if(same)
    printf("\n90 Degree Rotation is identical to Original Matrix.");
else
    printf("\nNo rotated matrix is identical to Original Matrix.");
return 0;
}

```

Output

```
main.c
1 #include <stdio.h>
2 int main()
3 {
4     int n, i, j;
5     int a[10][10];
6     int r90[10][10], r180[10][10], r270[10][10];
7     int same = 1;
8     printf("Enter order of matrix: ");
9     scanf("%d", &n);
10    printf("Enter matrix elements:\n");
11    for(i = 0; i < n; i++)
12        for(j = 0; j < n; j++)
13            scanf("%d", &a[i][j]);
14    for(i = 0; i < n; i++)
15        for(j = 0; j < n; j++)
16            r90[j][n - 1 - i] = a[i][j];
17    for(i = 0; i < n; i++)
18        for(j = 0; j < n; j++)
19            r180[j][n - 1 - i] = r90[i][j];
20    for(i = 0; i < n; i++)
21        for(j = 0; j < n; j++)
22            r270[j][n - 1 - i] = r180[i][j];
23    printf("\n90 Degree Rotation\n");
24    for(i = 0; i < n; i++)
25    {
26        for(j = 0; j < n; j++)
27            printf("%d ", r90[i][j]);
28        printf("\n");
29    }
30    printf("\n180 Degree Rotation\n");
31    for(i = 0; i < n; i++)
32    {
33        for(j = 0; j < n; j++)
34            printf("%d ", r180[i][j]);
35    }
```

```
Output
Enter order of matrix: 3
Enter matrix elements:
1 5 9
3 5 7
8 5 2

90 Degree Rotation
8 3 1
5 5 5
2 7 9

180 Degree Rotation
2 5 8
7 5 3
9 5 1

270 Degree Rotation
9 7 2
5 5 5
1 3 8

No rotated matrix is identical to Original Matrix.

--- Code Execution Successful ---
```

22. String Operations

Aim

To develop a C program to perform various string operations such as finding the string length, reversing the string, checking whether it is a palindrome, counting vowels and consonants, and displaying the frequency of each character.

Program Code

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

int main()
{
    char str[100], rev[100];

    int i, len;

    int vowels = 0, consonants = 0;

    int freq[256] = {0};

    printf("Enter a string: ");
```

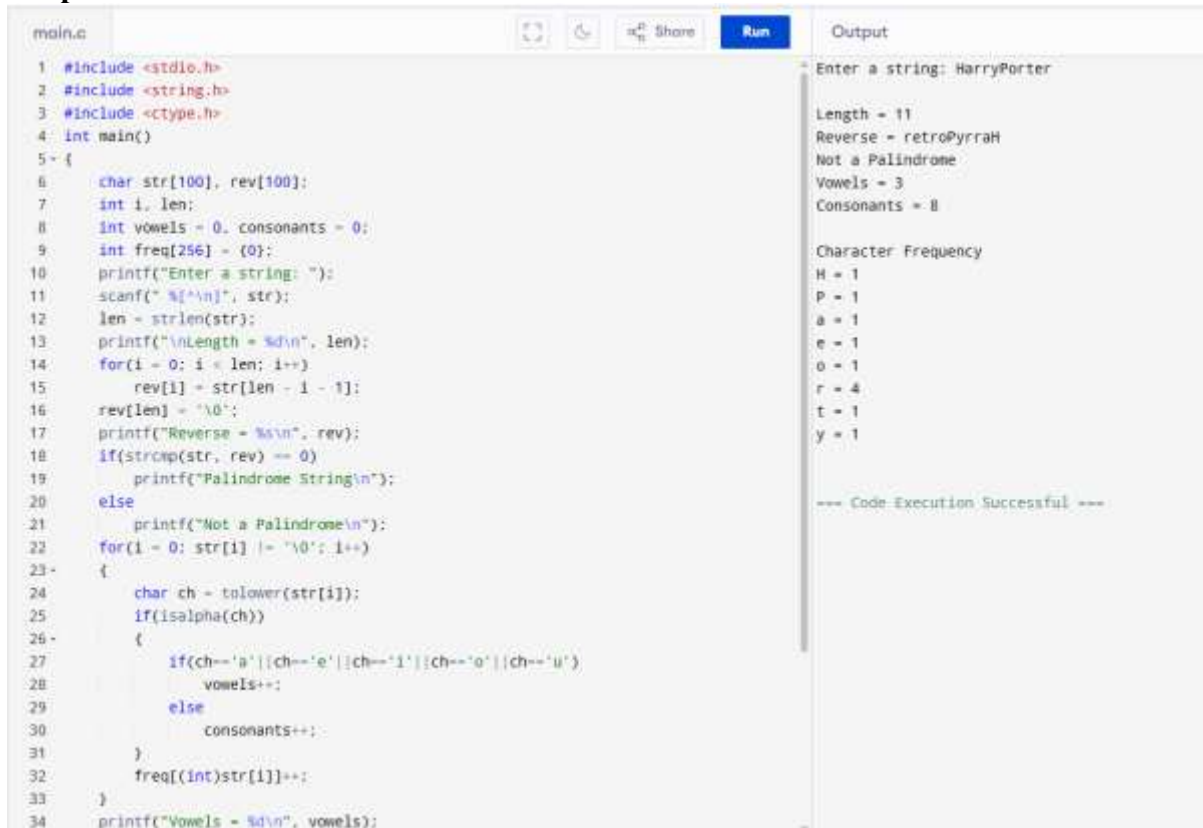
```

scanf("%[^\\n]", str);
len = strlen(str);
printf("\\nLength = %d\\n", len);
for(i = 0; i < len; i++)
    rev[i] = str[len - i - 1];
rev[len] = '\\0';
printf("Reverse = %s\\n", rev);
if(strcmp(str, rev) == 0)
    printf("Palindrome String\\n");
else
    printf("Not a Palindrome\\n");
for(i = 0; str[i] != '\\0'; i++)
{
    char ch = tolower(str[i]);
    if(isalpha(ch))
    {
        if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
            vowels++;
        else
            consonants++;
    }
    freq[(int)str[i]]++;
}
printf("Vowels = %d\\n", vowels);
printf("Consonants = %d\\n", consonants);
printf("\\nCharacter Frequency\\n");
for(i = 0; i < 256; i++)
{
    if(freq[i] > 0)
        printf("%c = %d\\n", i, freq[i]);
}
return 0;

```

}

Output



The screenshot shows a C program in a code editor with a 'Run' button. The code calculates the length, reverse, and character frequency of the string 'HarryPorter'. The output window shows the results of these calculations.

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 int main()
5 {
6     char str[100], rev[100];
7     int i, len;
8     int vowels = 0, consonants = 0;
9     int freq[256] = {0};
10    printf("Enter a string: ");
11    scanf("%s", str);
12    len = strlen(str);
13    printf("\nLength = %d\n", len);
14    for(i = 0; i < len; i++)
15        rev[i] = str[len - i - 1];
16    rev[i] = '\0';
17    printf("Reverse = %s\n", rev);
18    if(strcmp(str, rev) == 0)
19        printf("Palindrome String\n");
20    else
21        printf("Not a Palindrome\n");
22    for(i = 0; str[i] != '\0'; i++)
23    {
24        char ch = tolower(str[i]);
25        if(isalpha(ch))
26        {
27            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
28                vowels++;
29            else
30                consonants++;
31        }
32        freq[(int)str[i]]++;
33    }
34    printf("Vowels = %d\n", vowels);
```

Output

```
Enter a string: HarryPorter

Length = 11
Reverse = retroPyrraH
Not a Palindrome
Vowels = 3
Consonants = 8

Character Frequency
H = 1
P = 1
a = 1
e = 1
o = 1
r = 4
t = 1
y = 1

--- Code Execution Successful ---
```

23. String Compression

Aim

To develop a C program to compress a string using Run-Length Encoding (RLE), calculate the compression ratio, and display whether compression is beneficial.

Program Code

```
#include <stdio.h>

#include <string.h>

int main()
{
    char str[100], compressed[200];
    int i, j = 0, count;
    printf("Enter a string: ");
    scanf("%s", str);
    for(i = 0; str[i] != '\0'; i++)
```

```
{
    count = 1;
    while(str[i] == str[i+1])
    {
        count++;
        i++;
    }
    compressed[j++] = str[i];
    if(count > 1)
        j += sprintf(&compressed[j], "%d", count);
}
compressed[j] = '\0';
printf("\nOriginal String : %s\n", str);
printf("Compressed String : %s\n", compressed);
printf("Original Length = %d\n", strlen(str));
printf("Compressed Length = %d\n", strlen(compressed));
if(strlen(compressed) < strlen(str))
    printf("Compression is Beneficial\n");
else
    printf("Compression is Not Beneficial\n");
return 0;
}
```

Output


```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char str[100], compressed[200];
6     int i, j = 0, count;
7     printf("Enter a string: ");
8     scanf("%s", str);
9     for(i = 0; str[i] != '\0'; i++)
10     {
11         count = 1;
12         while(str[i] == str[i+1])
13         {
14             count++;
15             i++;
16         }
17         compressed[j++] = str[i];
18         if(count > 1)
19             j += sprintf(&compressed[j], "%d", count);
20     }
21     compressed[j] = '\0';
22     printf("Original String : %s\n", str);
23     printf("Compressed String : %s\n", compressed);
24     printf("Original Length = %d\n", strlen(str));
25     printf("Compressed Length = %d\n", strlen(compressed));
26     if(strlen(compressed) < strlen(str))
27         printf("Compression is Beneficial\n");
28     else
29         printf("Compression is Not Beneficial\n");
30     return 0;
31 }
```

Output

```
Enter a string: Stranger Things
Original String : Stranger
Compressed String : Stranger
Original Length = 8
Compressed Length = 8
Compression is Not Beneficial
--- Code Execution Successful ---
```

24. Matrix Search

Aim

To develop a C program to search an element in a matrix, display its position(s), count the number of occurrences, and identify the row and column having the maximum sum.

Program Code

```
#include <stdio.h>

int main()
{
    int a[10][10];
    int r, c;
    int i, j;
    int key;
    int count = 0;
    printf("Enter rows and columns: ");
    scanf("%d%d", &r, &c);
    printf("Enter matrix elements:\n");
    for(i=0; i<r; i++)
        for(j=0; j<c; j++)
            scanf("%d", &a[i][j]);
```

```

printf("Enter element to search: ");
scanf("%d",&key);
printf("\nPositions Found\n");
for(i=0;i<r;i++)
{
    for(j=0;j<c;j++)
    {
        if(a[i][j]==key)
        {
            printf("(%d,%d)\n",i,j);
            count++;
        }
    }
}
printf("Occurrences = %d\n",count);
int maxRow=0,maxCol=0,rowSum,colSum,maxRowSum=-9999,maxColSum=-9999;
for(i=0;i<r;i++)
{
    rowSum=0;
    for(j=0;j<c;j++)
        rowSum+=a[i][j];
    if(rowSum>maxRowSum)
    {
        maxRowSum=rowSum;
        maxRow=i;
    }
}
for(j=0;j<c;j++)
{
    colSum=0;
    for(i=0;i<r;i++)
        colSum+=a[i][j];
}

```

```

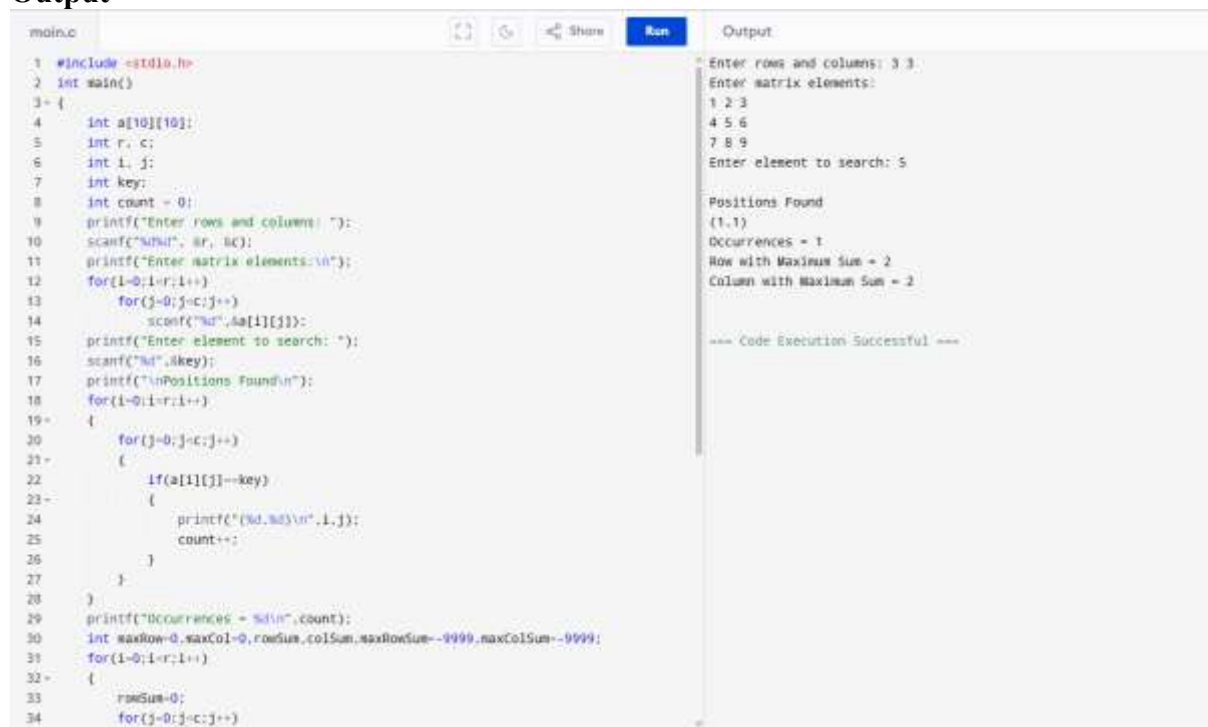
        if(colSum>maxColSum)
        {
            maxColSum=colSum;
            maxCol=j;
        }
    }

    printf("Row with Maximum Sum = %d\n",maxRow);
    printf("Column with Maximum Sum = %d\n",maxCol);

    return 0;
}

```

Output



The screenshot shows a C program in a code editor with a 'Run' button. The code is a 2D array sum calculator. The output window shows the program's execution: it prompts for rows and columns (3 and 3), then for matrix elements (1 2 3, 4 5 6, 7 8 9), then for an element to search (5). It then displays the positions found (1,1), the number of occurrences (1), the row with the maximum sum (2), and the column with the maximum sum (2). The execution is successful.

```

main.c
1 #include <stdio.h>
2 int main()
3 {
4     int a[10][10];
5     int r, c;
6     int i, j;
7     int key;
8     int count = 0;
9     printf("Enter rows and columns: ");
10    scanf("%d%d", &r, &c);
11    printf("Enter matrix elements:\n");
12    for(i=0; i<r; i++)
13        for(j=0; j<c; j++)
14            scanf("%d", &a[i][j]);
15    printf("Enter element to search: ");
16    scanf("%d", &key);
17    printf("\nPositions Found\n");
18    for(i=0; i<r; i++)
19    {
20        for(j=0; j<c; j++)
21        {
22            if(a[i][j]==key)
23            {
24                printf("(%d,%d)\n", i, j);
25                count++;
26            }
27        }
28    }
29    printf("Occurrences = %d\n", count);
30    int maxRow=0, maxCol=0, rowSum, colSum, maxRowSum=-9999, maxColSum=-9999;
31    for(i=0; i<r; i++)
32    {
33        rowSum=0;
34        for(j=0; j<c; j++)

```

```

Output
Enter rows and columns: 3 3
Enter matrix elements:
1 2 3
4 5 6
7 8 9
Enter element to search: 5

Positions Found
(1,1)
Occurrences = 1
Row with Maximum Sum = 2
Column with Maximum Sum = 2

--- Code Execution Successful ---

```

25. Duplicate Pattern Detector

Aim

To develop a C program to identify duplicate values in an array, display their frequencies, and print the repeated elements.

Program Code

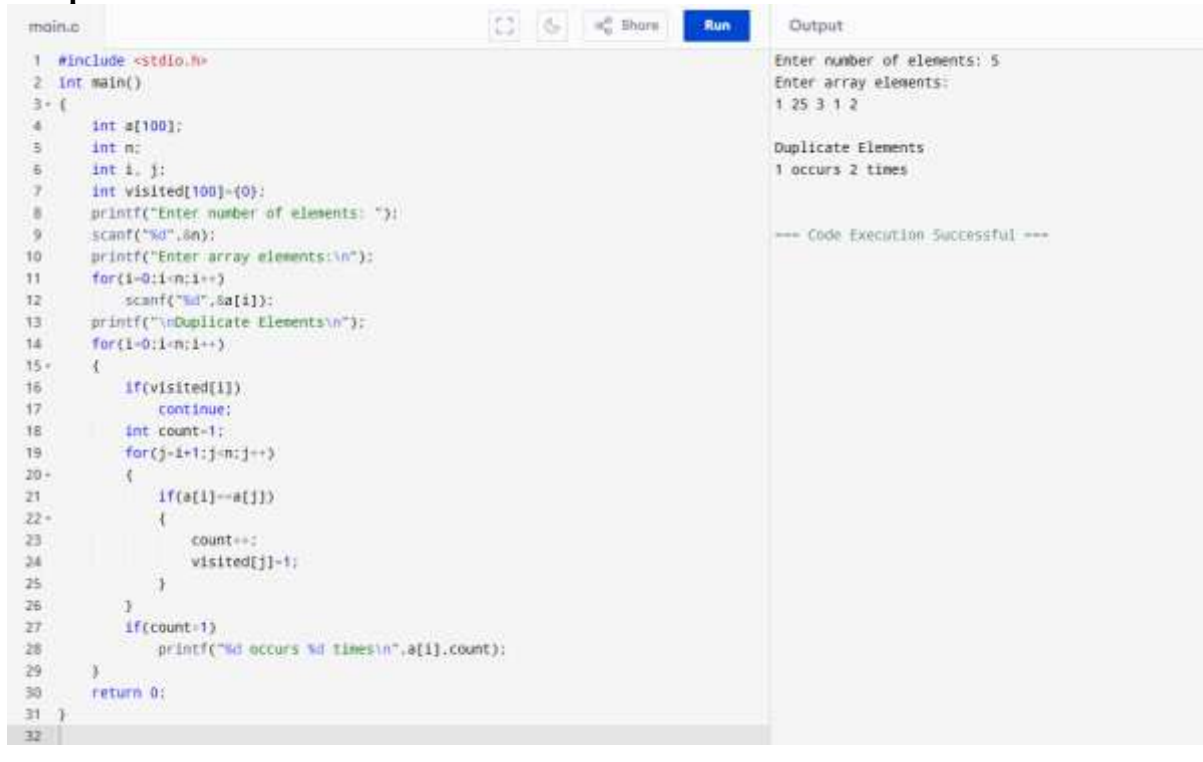
```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a[100];
int n;
int i, j;
int visited[100]={0};
printf("Enter number of elements: ");
scanf("%d",&n);
printf("Enter array elements:\n");
for(i=0;i<n;i++)
    scanf("%d",&a[i]);
printf("\nDuplicate Elements\n");
for(i=0;i<n;i++)
{
    if(visited[i])
        continue;
    int count=1;
    for(j=i+1;j<n;j++)
    {
        if(a[i]==a[j])
        {
            count++;
            visited[j]=1;
        }
    }
    if(count>1)
        printf("%d occurs %d times\n",a[i],count);
}
return 0;
}
```

Output



The screenshot shows a C program in a code editor with a 'Run' button. The code is as follows:

```
1 #include <stdio.h>
2 int main()
3 {
4     int a[100];
5     int n;
6     int i, j;
7     int visited[100]={0};
8     printf("Enter number of elements: ");
9     scanf("%d",&n);
10    printf("Enter array elements:\n");
11    for(i=0;i<n;i++)
12        scanf("%d",&a[i]);
13    printf("\nDuplicate Elements\n");
14    for(i=0;i<n;i++)
15    {
16        if(visited[i])
17            continue;
18        int count=1;
19        for(j=i+1;j<n;j++)
20        {
21            if(a[i]==a[j])
22            {
23                count++;
24                visited[j]=1;
25            }
26        }
27        if(count>1)
28            printf("%d occurs %d times\n",a[i],count);
29    }
30    return 0;
31 }
```

The output on the right shows the program's execution:

```
Enter number of elements: 5
Enter array elements:
1 25 3 1 2

Duplicate Elements
1 occurs 2 times

--- Code Execution Successful ---
```

26. Employee Database Search Engine

Aim

To develop a C program using structures to store employee records, search employees by ID or Name, display department-wise employees, and calculate the average salary of each department.

Program Code

```
#include <stdio.h>
#include <string.h>

struct Employee
{
    int id;
    char name[30];
    char dept[20];
    float salary;
};

int main()
```

```

{
    struct Employee e[50];
    int n,i,id;
    char name[30],dept[20];
    printf("Enter number of employees: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEmployee %d\n",i+1);
        printf("ID: ");
        scanf("%d",&e[i].id);
        printf("Name: ");
        scanf("%s",e[i].name);
        printf("Department: ");
        scanf("%s",e[i].dept);
        printf("Salary: ");
        scanf("%f",&e[i].salary);
    }
    printf("\nEnter Employee ID to search: ");
    scanf("%d",&id);
    for(i=0;i<n;i++)
    {
        if(e[i].id==id)
            printf("%d %s %s %.2f\n",e[i].id,e[i].name,e[i].dept,e[i].salary);
    }
    printf("\nEnter Employee Name to search: ");
    scanf("%s",name);
    for(i=0;i<n;i++)
    {
        if(strcmp(name,e[i].name)==0)
            printf("%d %s %s %.2f\n",e[i].id,e[i].name,e[i].dept,e[i].salary);
    }
}

```

```
printf("\nEnter Department: ");
scanf("%s",dept);
float sum=0;
int count=0;
printf("\nEmployees in %s Department\n",dept);
for(i=0;i<n;i++)
{
    if(strcmp(dept,e[i].dept)==0)
    {
        printf("%s %.2f\n",e[i].name,e[i].salary);
        sum+=e[i].salary;
        count++;
    }
}
if(count>0)
    printf("Average Salary = %.2f\n",sum/count);
return 0;
}
```

Output

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 struct Employee
4 {
5     int id;
6     char name[30];
7     char dept[20];
8     float salary;
9 };
10 int main()
11 {
12     struct Employee e[50];
13     int n,i,id;
14     char name[30],dept[20];
15     printf("Enter number of employees: ");
16     scanf("%d",&n);
17     for(i=0;i<n;i++)
18     {
19         printf("Enter Employee %d\n",i+1);
20         printf("ID: ");
21         scanf("%d",&e[i].id);
22         printf("Name: ");
23         scanf("%s",e[i].name);
24         printf("Department: ");
25         scanf("%s",e[i].dept);
26         printf("Salary: ");
27         scanf("%f",&e[i].salary);
28     }
29     printf("Enter Employee ID to search: ");
30     scanf("%d",&id);
31     for(i=0;i<n;i++)
32     {
33         if(e[i].id==id)
34             printf("%d %s %s %.2f\n",e[i].id,e[i].name,e[i].dept,e[i].salary);
35     }
36 }
```

Output

```
Enter number of employees: 2

Employee 1
ID: 192425
Name: Shaik
Department: AIML
Salary: 500000

Employee 2
ID: 1924
Name: Syed
Department: AIML
Salary: 425047

Enter Employee ID to search: 192425
192425 Shaik AIML 500000.00

Enter Employee Name to search: Shaik
192425 Shaik AIML 500000.00

Enter Department: CSE
Employees in CSE Department

--- Code Execution Successful ---
```

27. Palindrome Transformation

Aim

To develop a C program to determine whether a string is a palindrome and, if not, display the minimum number of character changes required to convert it into a palindrome.

Program Code

```
#include <stdio.h>

#include <string.h>

int main()
{
    char str[100];
    int i,len,count=0;
    printf("Enter a string: ");
    scanf("%s",str);
    len=strlen(str);
    for(i=0;i<len/2;i++)
    {
        if(str[i]!=str[len-1-i])
            count++;
    }
}
```



```

    if(count==0)

        printf("Palindrome String\n");

    else

        printf("Minimum Changes Required = %d\n",count);

    return 0;

}

```

Output

The screenshot shows a C program in a code editor with a 'Run' button. The code is as follows:

```

1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char str[100];
6     int i,len,count=0;
7     printf("Enter a string: ");
8     scanf("%s",str);
9     len=strlen(str);
10    for(i=0;i<len/2;i++)
11    {
12        if(str[i]!=str[len-1-i])
13            count++;
14    }
15    if(count==0)
16        printf("Palindrome String\n");
17    else
18        printf("Minimum Changes Required = %d\n",count);
19    return 0;
20 }

```

The output on the right shows the program's execution for the input 'Hello':

```

Enter a string: Hello
Minimum Changes Required = 2

```

Below the output, it says '*** Code Execution Successful ***'.

28. Matrix Boundary and Diagonal Analysis

Aim

To develop a C program to calculate the sum of boundary elements, principal diagonal, secondary diagonal, and determine whether the matrix is a magic square.

Program Code

```

#include <stdio.h>

int main()
{
    int a[10][10];
    int n,i,j;
    int boundary=0,pdiag=0,sdiag=0;
    printf("Enter order of matrix: ");
    scanf("%d",&n);
    printf("Enter matrix elements:\n");

```

```

for(i=0;i<n;i++)
    for(j=0;j<n;j++)
        scanf("%d",&a[i][j]);
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        if(i==0||j==0||i==n-1||j==n-1)
            boundary+=a[i][j];
        if(i==j)
            pdiag+=a[i][j];
        if(i+j==n-1)
            sdiag+=a[i][j];
    }
}
printf("\nBoundary Sum = %d\n",boundary);
printf("Principal Diagonal Sum = %d\n",pdiag);
printf("Secondary Diagonal Sum = %d\n",sdiag);
if(pdiag==sdiag)
    printf("Possible Magic Square\n");
else
    printf("Not a Magic Square\n");
return 0;
}

```

Output

main.c	Output
<pre> 1 #include <stdio.h> 2 int main() 3 { 4 int a[10][10]; 5 int n,i,j; 6 int boundary=0,pdiag=0,sdiag=0; 7 printf("Enter order of matrix: "); 8 scanf("%d",&n); 9 printf("Enter matrix elements:\n"); 10 for(i=0;i<n;i++) 11 for(j=0;j<n;j++) 12 scanf("%d",&a[i][j]); 13 for(i=0;i<n;i++) 14 { 15 for(j=0;j<n;j++) 16 { 17 if(i==0 j==0 i==n-1 j==n-1) 18 boundary+=a[i][j]; 19 if(i==j) 20 pdiag+=a[i][j]; 21 if(i+j==n-1) 22 sdiag+=a[i][j]; 23 } 24 } 25 printf("\nBoundary Sum = %d\n",boundary); 26 printf("Principal Diagonal Sum = %d\n",pdiag); 27 printf("Secondary Diagonal Sum = %d\n",sdiag); 28 if(pdiag==sdiag) 29 printf("Possible Magic Square\n"); 30 else 31 printf("Not a Magic Square\n"); 32 return 0; 33 } 34 </pre>	<pre> Enter order of matrix: 2 Enter matrix elements: 1 2 3 4 Boundary Sum = 10 Principal Diagonal Sum = 5 Secondary Diagonal Sum = 5 Possible Magic Square === Code Execution Successful === </pre>

29. Intelligent Dictionary Manager

Aim

To develop a C program using structures to store words and their meanings, search for a word, display all dictionary entries in alphabetical order, and count the total number of words.

Program Code

```

#include <stdio.h>
#include <string.h>
struct Dictionary
{
    char word[30];
    char meaning[100];
};
int main()
{
    struct Dictionary d[50],temp;
    int n,i,j;
    char search[30];
    printf("Enter number of words: ");

```

```

scanf("%d",&n);
for(i=0;i<n;i++)
{
    printf("\nWord: ");
    scanf("%s",d[i].word);
    printf("Meaning: ");
    scanf(" %[^\\n]",d[i].meaning);
}
printf("\nEnter word to search: ");
scanf("%s",search);
for(i=0;i<n;i++)
{
    if(strcmp(search,d[i].word)==0)
    {
        printf("\nMeaning : %s\\n",d[i].meaning);
        break;
    }
}
for(i=0;i<n-1;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(strcmp(d[i].word,d[j].word)>0)
        {
            temp=d[i];
            d[i]=d[j];
            d[j]=temp;
        }
    }
}
printf("\nDictionary Entries\\n");
for(i=0;i<n;i++)

```

```

        printf("%s : %s\n",d[i].word,d[i].meaning);
    printf("\nTotal Words = %d\n",n);
    return 0;
}

```

Output

The screenshot shows a C++ IDE with a code editor on the left and an output window on the right. The code in the editor is as follows:

```

1 #include <stdio.h>
2 #include <string.h>
3 struct Dictionary
4 {
5     char word[30];
6     char meaning[100];
7 };
8 int main()
9 {
10     struct Dictionary d[50].temp;
11     int n,i,j;
12     char search[30];
13     printf("Enter number of words: ");
14     scanf("%d",&n);
15     for(i=0;i<n;i++)
16     {
17         printf("word: ");
18         scanf("%s",d[i].word);
19         printf("Meaning: ");
20         scanf("%s",d[i].meaning);
21     }
22     printf("\nEnter word to search: ");
23     scanf("%s",search);
24     for(i=0;i<n;i++)
25     {
26         if(strcmp(search,d[i].word)==0)
27         {
28             printf("\nMeaning : %s",d[i].meaning);
29             break;
30         }
31     }
32     for(i=0;i<n-1;i++)
33     {
34         for(j=i+1;j<n;j++)

```

The output window on the right shows the following text:

```

Enter number of words: 3

Word: Shaik
Meaning: name

Word: Mubarak
Meaning: Wishes

Word: Mango
Meaning: fruit

Enter word to search: Wishes

Dictionary Entries
Mango : fruit
Mubarak : Wishes
Shaik : name

Total Words = 3

=== Code Execution Successful ===

```